



# Graph Neural Networks for Personalisation

Man Kit Chan<sup>1</sup>

MSc Computational Statistics and Machine Learning

Internal Supervisor:  
Dmitry Adamskiy, UCL

Industry Supervisors:  
Shaun Read, Tesco  
Chris Wilkin, Tesco

Submission Date:  
9<sup>th</sup> September 2024

<sup>1</sup>**Disclaimer:** This report is submitted as part requirement for the MSc Computational Statistics and Machine Learning at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report may be freely copied and distributed provided the source is explicitly acknowledged.

# Abstract

This thesis explores the potential of Graph Neural Networks (GNNs) in enhancing personalisation quality of recommender systems. Traditional Collaborative Filtering (CF) methods like Matrix Factorisation (MF), often fail to capture the full relationships between users and items due to convoluted interactions and data sparsity. This study aims to investigate whether GNN-based models can further improve recommendation models by leveraging structural information from the user-item interaction graph. The scalability and efficiency of GNN-based models on large-scale retail datasets are also assessed, as well as the graph embeddings quality and the overall recommendation effectiveness.

We have demonstrated that GNN-based approaches such as Neural Graph Collaborative Filtering (NGCF) and Light Graph Convolution Network (LightGCN) surpass traditional methods in terms of ranking performance and downstream recommendation tasks. Also, we have proposed and evaluated two inference strategies which offer good scalability to large graphs, with one being a hybrid framework integrating graph SAMpling and agGrEgation (graphSAGE) to transductive GNN-based models such as NGCF and LightGCN. The findings contribute to future development of more sophisticated recommendation systems, bridging the gap of GNN-based recommendation models between theoretical advancements and practical applications for industry.

# Acknowledgements

I would like to sincerely thank my internal supervisor, Dmitry Adamskiy, for his continuous support and guidance throughout this project. His oversight in managing the project's progress and timeline ensured that I remained on track and focused. I also want express my deep gratitude to my industry supervisors, Shaun Read and Chris Wilkin at Tesco, for their technical expertise and insightful comments. Their direction in shaping the research and their technical input were critical to the development and success of this project. Finally, thanks to all my colleagues at Tesco as well as the broader Machine Learning community at UCL for their support and encouragement.

# Contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                               | <b>5</b>  |
| 1.1      | Background . . . . .                                              | 5         |
| 1.2      | Preliminary Knowledge . . . . .                                   | 6         |
| 1.2.1    | Recommender Systems . . . . .                                     | 6         |
| 1.2.2    | Collaborative Filtering . . . . .                                 | 8         |
| 1.2.3    | Graph Representation Learning . . . . .                           | 12        |
| 1.3      | Related Work . . . . .                                            | 17        |
| 1.3.1    | Graph Representation Learning and Spectral Convolutions . . . . . | 17        |
| 1.3.2    | Graph-Based Recommendation Models . . . . .                       | 18        |
| 1.3.3    | GNN-Based Recommendation Models . . . . .                         | 19        |
| 1.3.4    | State-of-the-Art GNN-Based Recommendation Models . . . . .        | 19        |
| 1.4      | Problem Statement . . . . .                                       | 20        |
| 1.5      | Research Objectives . . . . .                                     | 20        |
| 1.6      | Significance of the Study . . . . .                               | 20        |
| <b>2</b> | <b>Methodology</b>                                                | <b>22</b> |
| 2.1      | Datasets . . . . .                                                | 22        |
| 2.1.1    | Data Sources . . . . .                                            | 22        |
| 2.1.2    | Data Preprocessing . . . . .                                      | 22        |
| 2.1.3    | Experimental Datasets . . . . .                                   | 26        |
| 2.2      | Model Architectures . . . . .                                     | 28        |
| 2.2.1    | Neural Graph Collaborative Filtering (NGCF) . . . . .             | 28        |
| 2.2.2    | Light Graph Convolution Networks (LightGCN) . . . . .             | 30        |
| 2.2.3    | Graph SAmple and aggreGatE (GraphSAGE) . . . . .                  | 31        |
| 2.3      | Experimental Setup . . . . .                                      | 34        |
| 2.3.1    | Optimisation . . . . .                                            | 34        |
| 2.3.2    | Model Hyperparameters . . . . .                                   | 35        |
| 2.3.3    | Large-scale Model Training and Inference . . . . .                | 36        |
| 2.4      | Evaluation . . . . .                                              | 37        |
| 2.4.1    | Ranking Metrics . . . . .                                         | 37        |
| 2.4.2    | Downstream Tasks . . . . .                                        | 38        |
| 2.4.3    | Embedding Space Analysis . . . . .                                | 39        |
| 2.4.4    | Large-scale Training Analysis . . . . .                           | 40        |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>3</b> | <b>Result Analysis</b>                         | <b>41</b> |
| 3.1      | Reproducing Yelp2018 Ranking Metrics . . . . . | 41        |
| 3.2      | Hyperparameter Search . . . . .                | 41        |
| 3.3      | Ranking Performance . . . . .                  | 42        |
| 3.3.1    | Tesco-20k . . . . .                            | 42        |
| 3.3.2    | Tesco-228k . . . . .                           | 43        |
| 3.4      | Downstream Tasks Performance . . . . .         | 44        |
| 3.5      | Embedding Space Analysis . . . . .             | 45        |
| 3.5.1    | Clustering Analysis . . . . .                  | 45        |
| 3.5.2    | Sparsity Analysis . . . . .                    | 49        |
| 3.6      | Scalability Analysis . . . . .                 | 50        |
| 3.6.1    | Training Strategies Analysis . . . . .         | 50        |
| 3.6.2    | Training Speed Analysis . . . . .              | 51        |
| <b>4</b> | <b>Method Extension</b>                        | <b>53</b> |
| 4.1      | More Large-Scale Training Techniques . . . . . | 53        |
| 4.2      | Incorporating Node Features . . . . .          | 53        |
| <b>5</b> | <b>Conclusion</b>                              | <b>56</b> |
| <b>A</b> | <b>Detailed Ranking Metrics</b>                | <b>58</b> |
| <b>B</b> | <b>Detailed Embedding Analysis</b>             | <b>60</b> |
|          | <b>Bibliography</b>                            | <b>58</b> |

# List of Figures

|      |                                                                                                                                                                                                                                                |    |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1  | High-level diagram of NCF (left) and the Two-Tower Model (right).                                                                                                                                                                              | 12 |
| 1.2  | Bipartite graph representation of toy movie ratings example in Table 1.1.                                                                                                                                                                      | 13 |
| 1.3  | Visual aid of GCN traversing a graph for message passing, figure materials sourced from Daigavane et al. [?] ]                                                                                                                                 | 17 |
| 2.1  | Node degree distributions for different k-core filtering settings. (a) Raw data; (b) $k_{\text{user}} = 350$ , $k_{\text{item}} = 350$ filtered data; (c) $k_{\text{user}} = 358$ , $k_{\text{item}} = 21$ filtered data.                      | 25 |
| 2.2  | NGCF architecture overview sourced from the original paper [? ].                                                                                                                                                                               | 30 |
| 2.3  | LightGCN architecture overview sourced from the original paper [? ].                                                                                                                                                                           | 31 |
| 2.4  | Visualisation of the GraphSAGE sample and aggregate mechanism from the original paper [? ].                                                                                                                                                    | 32 |
| 2.5  | High-level diagram of the Personalised Offers downstream model.                                                                                                                                                                                | 38 |
| 3.1  | Tesco-20k test set ranking metrics of different models trained until convergence.                                                                                                                                                              | 43 |
| 3.2  | Tesco-228k test set ranking metrics of different models trained for 10 epochs.                                                                                                                                                                 | 44 |
| 3.3  | Comparison of personalised offers performance across different models.                                                                                                                                                                         | 45 |
| 3.4  | Mean Silhouette score by assigning item embeddings into categories according to Tesco commercial hierarchy (a, left), and clustering item embeddings using k-means (b, right). Item embeddings are generated by LightGCN trained on Tesco-20k. | 46 |
| 3.5  | Silhouette score per sample sorted in descending order, by assigning item embeddings into categories according to Tesco commercial hierarchy level 1. Item embeddings are generated by LightGCN trained on Tesco-20k.                          | 46 |
| 3.6  | UMAP of item embeddings of category 0 and 3 from hierarchy level 1, generated by LightGCN trained on Tesco-20k. The decision boundary of the two regions is learnt by a 2-component Gaussian Mixture Model.                                    | 47 |
| 3.7  | Mean Silhouette score by clustering user embeddings using k-means. Item embeddings are generated by LightGCN trained on Tesco-20k.                                                                                                             | 48 |
| 3.8  | Mean Silhouette score by clustering user embeddings using k-means. Item embeddings are generated by LightGCN trained on Tesco-20k.                                                                                                             | 48 |
| 3.9  | Magnitude of each dimension in the mean item embeddings (512-d) of category 0 and 3 from hierarchy level 1, generated by LightGCN trained on Tesco-20k.                                                                                        | 49 |
| 3.10 | Magnitude of each dimension in the mean user embeddings (512-d), generated by LightGCN trained on Tesco-20k.                                                                                                                                   | 50 |

|      |                                                                                                                                                                                                                                    |    |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.11 | Max normalised training loss curves comparing different epoch definitions (a, top), and Rescaled Recall@20 curve on validation set comparing different epoch definitions (b, bottom) . . . . .                                     | 51 |
| 3.12 | Average computation time (seconds) per batch across different batch sizes and inference strategies. . . . .                                                                                                                        | 52 |
| 4.1  | Graph Convolutional Network architecture with GraphSAGE mean aggregator for user and item categorical features input. . . . .                                                                                                      | 54 |
| 4.2  | Heterogenous NGCF architecture for item fastText features input. . . . .                                                                                                                                                           | 55 |
| B.1  | UMAP of item embeddings of all categories from Tesco commercial hierarchy level 1, generated by LightGCN trained on Tesco-20k. The decision boundary of the two regions is learnt by a 2-component Gaussian Mixture Model. . . . . | 60 |
| B.2  | Magnitude of each dimension in the mean item embeddings (512-d) of all categories from Tesco commercial hierarchy level 1, generated by LighGCN trained on Tesco-20k. . . . .                                                      | 62 |

# Chapter 1

## Introduction

### 1.1 Background

In today’s digital age, users are facing great amount of choices across different online platforms such as e-commerce websites and streaming services. While modern online platform provides an enormous array of products and content to users, it also presents a significant challenge: how can users efficiently discover products that truly align with their preferences and needs? This is where recommender systems come into play, serving as an essential tool to help users navigate this vast sea of choices.

Recommender systems have become integral to many services, including e-commerce platforms, entertainment streaming services, social media networks and even professional services, such as research article exploration and financial decision-making. These systems analyse historical user-item interaction data to predict user preferences and suggest items likely to be relevant. By providing personalised recommendations, they enhance user experience and drive engagement, ultimately contribute to the success of many digital platforms.

The core challenge in developing effective recommender systems lies in accurately modeling user preferences and item characteristics based on limited observations. This task is particularly complex due to the sparse nature of user-item interaction data that most users interact with a tiny fraction of available items. Moreover, user preferences are often dynamic and context-dependent which add another layer of complexity to the prediction task.

Traditional approaches to recommendation such as content-based filtering and Collaborative Filtering (CF) have shown success as well as its shortcoming. Content-based methods rely on item features that they may struggle with complex items like movies or music features cannot easily defined. CF methods, such as Matrix Factorisation (MF), discover hidden patterns in user behavior but may suffer from the “cold start” problem for new users or items, it is also difficult to capture the full spectrum of user-item interactions in the latent space [? ].

The evolution of graph-based approaches in recommendation systems has been significant, leading to the development of various specialised models like the Collaborative Memory Network (CMN), Graph Convolutional Matrix Completion (GC-MC) and High-Order Proximity for implicit recommendation (HOP-Rec). CMN introduced a memory component and neural attention mechanism to capture complex user-item interactions [? ], GC-MC framed the matrix completion



problem as a link prediction task on a bipartite user-item graph [? ], HOP-Rec integrated latent factor models with random walking techniques on the sparse user-item interaction graph to obtain high-order proximity information [? ].

Recent advancements in the field of Graph Neural Networks (GNNs) especially its subclass Graph Convolutional Networks (GCNs) have provided a more general framework with stronger theoretical foundation. The development of Graph SAmple and aggreGatE (GraphSAGE) enables GCN-based inductive and transductive learning on large-scale graphs [? ]. Many of its variants like PinSage developed by Pinterest [? ], are leveraging the SAGE technique on GCNs for recommendation tasks. Many recent research has demonstrated the potential of GNN-based recommendation model for large-scale graphs [? ? ].

Our study focuses on investigating some of GNN-based recommendation model architectures, specifically Neural Graph Collaborative Filtering (NGCF) and Light Graph Convolution Network (LightGCN). These architectures adhere to the standard GNN framework with tailor-made convolution layers [? ? ]. The goal is to explore how well GNN-based models perform in capturing user preferences and item relationships and how they perform on large real-world datasets.

This research seeks to provide insights into the strengths and limitations of these models, their scalability to large datasets, and their potential for enhancing real-world recommendation systems. Ultimately this research could pave the way for more sophisticated, accurate and efficient recommender systems, improving the experience for millions of users across online platforms.

## 1.2 Preliminary Knowledge

### 1.2.1 Recommender Systems

Consider the setting of recommender system with a group of users and a group of items, where users have rated a subset of these items in the past. Our goal is to predict the ratings for the unrated subset of items for each user, thus determining user preferences for the unrated items. Recommendations can be made by ranking the predicted ratings. Definition 1 states the common notations used within a general recommender system framework to model ratings [? ].

**DEFINITION 1** *Set of all users is  $\mathcal{U}$ . Set of all items is  $\mathcal{I}$ . Set of all rating history is  $\mathcal{R}$ . Set of possible rating values is  $\mathcal{S}$ . The rating of item  $i$  from user  $u$  is  $r_{ui}$ . The set of users who rated item  $i$  is  $\mathcal{U}_i$ . The set of items rated by user  $u$  is  $\mathcal{I}_u$ .*

A general recommendation model can be defined as  $f : \mathcal{U} \times \mathcal{I} \rightarrow \mathcal{S}$  such that  $f(u, i) = \hat{r}_{ui}$  predicts the rating for any item  $i$  of any user  $u$  based on training data  $\mathcal{R}$ . This prediction task can be formulated as a regression, binary classification or multi-class classification problem depending on the nature  $\mathcal{S}$  and the model use case.

To find the top-N recommendations for any user  $u$ , we first compute the ratings  $\hat{r}_{ui}$  of all unrated items  $i \in \mathcal{I} \setminus \mathcal{I}_u$ , then select the N items with the highest predicted ratings. For the case of  $N = 1$ , the best item  $i^*$  for user  $u$  is given by:

$$i^* = \operatorname{argmax}_{j \in \mathcal{I} \setminus \mathcal{I}_u} f(u, j)$$

Recommender systems usually encounter scenarios where explicit rating records are insufficient or absent. In reality, most users are not motivated to rate items, or users often find most items too ordinary to put effort of giving positive or negative feedback. However, there are implicit feedbacks that the model can make good use of, for example, click patterns during browsing, watch length on video streaming platforms, add-to-cart or purchase actions on e-commerce platforms. These implicit feedbacks can be effective indicators for user behaviours and should be incorporated for better recommendation quality [? ].

## Ranking Metrics

There are multiple ranking metrics to evaluate a recommender system’s performance. Each metric provides unique insights into different aspects of the recommendation quality. In this study, we will be focusing on the following metrics.

**Recall@k** measures the proportion of relevant items that are successfully retrieved in the top-k recommendations.

$$\text{Recall@k} = \frac{\text{number of relevant items in top k recommendations}}{\text{total number of relevant items}}$$

This metric is useful for tracking whether most relevant items are included in the recommendations, regardless of their exact order.

**Precision@k** measures the proportion of recommended items in the top-k that are relevant.

$$\text{Precision@k} = \frac{\text{number of relevant items in top k recommendations}}{k}$$

This metric is useful for tracking whether most of the recommended items are relevant, regardless of the total number of relevant items.

**Rescaled Recall@k** is a hybrid metric that adapts based on the relationship between  $k$  and the total number of relevant items for a user.

$$\text{Rescaled Recall@k} = \frac{\text{number of relevant items in top k recommendations}}{\min(k, \text{total number of relevant items})}$$

This metric behaves like standard Recall@k when  $k < \text{number of relevant items}$  and it behaves like Precision@k when  $k \geq \text{number of relevant items}$ . The use of the  $\min(\cdot)$  function in the denominator automatically adjusts the metric based on whether we have more recommendations ( $k$ ) or more relevant items. We propose this metric since standard Recall@k can be misleading that it can be very low when total number of relevant items  $\gg k$ , even if the top-k recommendations are all relevant.

Rescaled Recall@k is particularly useful when comparing performance across datasets with different scales or characteristics. Its adaptive nature allows fair comparisons between recommender systems tested on datasets where the number of relevant items per user varies significantly. This makes it a valuable metric for evaluating recommendation quality consistently across diverse datasets and experimental settings.

**Average Precision@k (AP@k)** provides a single-value measure of quality across recall levels. It is calculated as the average of precisions at each relevant item in the top-k recommendations:

$$\text{AP@k} = \frac{\sum_{i=1}^k \text{Precision@i} \times \text{rel(i)}}{\text{number of relevant items in top k recommendations}}$$

where  $\text{rel(i)}$  is an indicator function, in our case it equals to 1 if the recommended item at rank  $i$  is relevant, zero otherwise.  $\text{AP@k}$  aggregates model precision at different recall levels and the order of recommendations. It gives more weight to relevant items appearing earlier in the recommendation list, which aligns well with user experience in many recommendation scenarios.

**Normalised Discounted Cumulative Gain (NDCG@k)** measures the quality of ranking by incorporating both the relevance and the position of relevant items in the recommendation list.

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}$$

where  $\text{DCG@k}$  (Discounted Cumulative Gain) is:

$$\text{DCG@k} = \sum_{i=1}^k \frac{2^{\text{rel(i)}} - 1}{\log_2(i + 1)}$$

and  $\text{IDCG@k}$  is the  $\text{DCG@k}$  of the ideal ranking.  $\text{NDCG@k}$  takes into account the graded relevance of recommendations and penalises highly relevant items appearing lower in the recommendation list.

Each metric provides a different perspective on the recommender system’s performance.  $\text{Recall@k}$  and  $\text{Precision@k}$  focus on the presence of relevant items while  $\text{AP@k}$  and  $\text{NDCG@k}$  consider the relative order of relevant items. Rescaled  $\text{Recall@k}$  offers a flexible approach that adapts to different scenarios based on the number of relevant items available. Note that all these metrics are computed per user thus yielding a distribution when we evaluate all users within the dataset. In practice, the mean of the distribution for each metric is taken to aggregate the overall performance, which is how the experiment results are presented. Still, it is worth to further check whether the distributions are well-behaved as to verify if the model is functioning properly.

## 1.2.2 Collaborative Filtering

### Neighbourhood Models

Neighbourhood models are based on the empirical word-of-mouth principle among users, where individuals rely on the opinions of like-minded users to decide which items to choose.

Consider a toy example on a movie streaming platform as shown in Table 1.1. Intuitively, Bob and Charlie have similar tastes because they both liked “Avengers: Endgame” and “The Dark Knight”, whereas Diana’s preferences appear to be the opposite. Therefore, if a neighbourhood model were to recommend new movies for Charlie, it would rely more heavily on Bob’s ratings than Diana’s that “John Wick” is more likely to be recommended than “Interstellar”. In practice, neighbourhood models calculate the similarity of all other users to the target user based on the current rating records, then determine the user’s neighbourhood and aggregates their ratings into

the final prediction for each movie. Another insight from this toy example is that we can frame the recommendation problem as a matrix completion problem by leveraging the available partial information.

|         | <b>John Wick</b> | <b>Avengers:<br/>Endgame</b> | <b>Inception</b> | <b>Interstellar</b> | <b>The Dark Knight</b> |
|---------|------------------|------------------------------|------------------|---------------------|------------------------|
| Alice   | 4                |                              | 5                | 2                   |                        |
| Bob     | 4                | 4                            |                  |                     | 5                      |
| Charlie |                  | 5                            | 3                |                     | 4                      |
| Diana   | 1                | 2                            |                  | 5                   | 1                      |
| Eve     | 2                |                              | 4                | 5                   |                        |

Table 1.1: User ratings for a movie streaming platform.

This user-based neighbourhood concept can be formulated as a weighted average of the k-nearest-neighbours (k-NN) of the target user  $u$ , i.e. the top  $k$  users  $v$  with preferences similar to  $u$ . To predict the rating  $r_{ui}$  of an item  $i$  that  $u$  has not seen, the estimator  $\hat{r}_{ui}$  can be computed as:

$$\hat{r}_{ui} = \frac{\sum_{v \in \mathcal{N}_i(u)} w_{uv} r_{vi}}{\sum_{v \in \mathcal{N}_i(u)} |w_{uv}|}$$

where  $\mathcal{N}_i(u)$  is the set of k-NN of  $u$  who have rated  $i$ ,  $w_{uv}$  is the similarity measure between user  $u$  and  $v$ . The denominator is the normalisation constant.

Note that users may have different rating habits, for example some users may be more lenient who tend to give higher ratings while some are critical who rarely give high rating unless the movie really entertains and impresses them. Therefore, rating normalisation can further be applied for each user before k-NN aggregation to adjust for individual rating habits. This can be written as:

$$\hat{r}_{ui} = h^{-1} \left( \frac{\sum_{v \in \mathcal{N}_i(u)} w_{uv} h(r_{vi})}{\sum_{v \in \mathcal{N}_i(u)} |w_{uv}|} \right)$$

where  $h(\cdot)$  is any normalisation function such as mean-centering normalisation or z-score normalisation.

Likewise, we can formulate an item-based neighbourhood model that considers the similarity between items, i.e., finding items which are usually liked by certain groups of users.

$$\hat{r}_{ui} = h^{-1} \left( \frac{\sum_{j \in \mathcal{N}_u(i)} w_{ij} h(r_{uj})}{\sum_{j \in \mathcal{N}_u(i)} |w_{ij}|} \right)$$

where  $\mathcal{N}_u(i)$  is the set of k-NN of  $i$  that  $u$  has rated,  $w_{ij}$  is the similarity measure between item  $i$  and  $j$ . This formulation considers other items  $j$  similar to  $i$  that  $u$  has rated and aggregate their ratings to make the final prediction [? ].

## Latent Factor Models

Latent factor models aim to identify hidden representations behind the observed ratings, with Matrix Factorisation (MF) models being the most popular due to their high accuracy and stability.

One standard MF model is the Singular Value Decomposition (SVD) algorithm:

$$\hat{r}_{ui} = \mu + b_i + b_u + \mathbf{q}_i^\top \mathbf{p}_u$$

In this model,  $\mu$  is the global average of all observed ratings,  $b_i$  represents the item bias which reflects the overall “quality” of the item and  $b_u$  represents the user bias which reflects the user’s general tendency to rate more critically or leniently.  $\mathbf{q}_i$  is a vector where each element captures the item’s characteristics in each latent factor and  $\mathbf{p}_u$  is a vector where each element represents the user’s interest in each latent factor.  $b_i$ ,  $b_u$ ,  $\mathbf{q}_i$  and  $\mathbf{p}_u$  are all learnable parameters.

One way to optimise the model parameters is through minimising the squared error with L2 regularisation:

$$\min_{b_i, b_u, \mathbf{q}_i, \mathbf{p}_u} \sum_{(u,i) \in \mathcal{K}} (r_{ui} - \mu - b_i - b_u - \mathbf{q}_i^\top \mathbf{p}_u)^2 + \lambda(b_i^2 + b_u^2 + \|\mathbf{q}_i\|^2 + \|\mathbf{p}_u\|^2)$$

where the set  $\mathcal{K} = \{(u, i) \in \mathcal{U} \times \mathcal{I} \mid r_{ui} \in \mathcal{R}\}$  represents all user-item pairs for which ratings are observed. The regularisation parameter  $\lambda$  is typically determined through cross-validation. This optimisation problem can be solved using Alternating Least Squares (ALS) or Stochastic Gradient Descent (SGD).

The ALS approach involves iteratively alternating between fixing all  $\mathbf{p}_u$  to solve for  $\mathbf{q}_i$  and fixing all  $\mathbf{q}_i$  to solve for  $\mathbf{p}_u$ . In each iteration, the objective function becomes a standard quadratic programming problem, which can be solved in parallel using an analytic solution.

The SGD approach is more commonly used in practice due to its ease of implementation. The following algorithm is widely used after being popularised by Simon Funk during the Netflix Prize Competition [? ]:

$$\begin{aligned} b_i &\leftarrow b_i + \alpha(e_{ui} - \lambda b_i) \\ b_u &\leftarrow b_u + \alpha(e_{ui} - \lambda b_u) \\ \mathbf{q}_i &\leftarrow \mathbf{q}_i + \alpha(e_{ui}\mathbf{p}_u - \lambda \mathbf{q}_i) \\ \mathbf{p}_u &\leftarrow \mathbf{p}_u + \alpha(e_{ui}\mathbf{q}_i - \lambda \mathbf{p}_u) \end{aligned}$$

Here  $e_{ui} := r_{ui} - \hat{r}_{ui}$  is the prediction error and  $\alpha$  is the learning rate. It is also worth to note that  $\alpha$  and  $\lambda$  can be set differently for each model parameter. Fine-tuning these hyperparameters for each model parameter may lead to performance improvement.

To further enhance the accuracy of SVD, implicit feedback can be incorporated to offer supplementary insights into user preferences. This is particularly advantageous for users who offer more implicit feedback than explicit feedback, significant signals can be captured from user rating behaviours, irrespective of their rating values. Below, we introduce the SVD++ algorithm:

$$\hat{r}_{ui} = \mu + b_i + b_u + \mathbf{q}_i^\top (\mathbf{p}_u + |I_u|^{-\frac{1}{2}} \sum_{j \in I_u} y_j)$$

where  $y_j$  is the learnable implicit ratings of the  $j$ -th item by user  $u$ . Implicit ratings can be any form of user feedback, such as clicks, add-to-cart actions, purchase history in e-commerce contexts, or viewing history in video streaming platforms [? ? ].

## Neural Collaborative Filtering

Advancements in deep learning have enabled non-linear modelling of latent factors, leading to Neural Collaborative Filtering (NCF) which utilises neural networks in capturing signals from user-item interactions. The general structure of NCF includes four components: input layer, embedding layer, neural collaborative layers and output layer.

The **input layer** receives input feature vectors representing user  $u$  and item  $i$ . In pure collaborative filtering scenarios, these are binary sparse vectors of one-hot encoded user and item identifiers. Additionally, side information of user and item can be incorporated to address the cold-start problem. The **embedding layer** converts the input feature vectors into dense embedding vectors, capturing latent representations in the context of latent factor models. The **neural collaborative layers** form the core of the neural network, where multiple hidden layers model the non-linear interactions between users and items. The **output layer** produces a single value, the predicted rating  $\hat{r}_{ui}$ , for the given user-item pair. The typical architecture within NCF components is a multi-layer perceptron (MLP) where hidden layers are fully connected, other architectures such as Convolution Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) can be adapted for input features of different data types.

The parameters of NCF are optimised like ordinary neural networks, by minimizing a loss function such as pointwise loss like squared loss, or pairwise loss like pairwise ranking loss, with regularisation to prevent overfitting.

Note that the vanilla MF model can be derived from the NCF framework by using a single neural CF layer which performs element-wise multiplication, an output layer with unit weights and a linear activation function. To allow maximum flexibility of MF, the Generalised Matrix Factorisation (GMF) model is proposed by He et al., which uses learnable weights and a non-linear activation function such as sigmoid function in the output layer [? ]. Below we will be introducing some of the widely used NCF models.

**Neural Matrix Factorisation (NeuMF)** combines the strengths of GMF and MLP, with GMF capturing linear latent structures and MLP capturing non-linear latent structures. Typically, GMF and MLP components are pretrained separately to ensure stable convergence and to learn different embeddings for greater flexibility. The NeuMF architecture is described by the following equation [? ]:

$$\hat{r}_{ui} = \sigma(\mathbf{h}^\top \begin{bmatrix} \mathbf{z}_u^{\text{GMF}} \\ \mathbf{z}_u^{\text{MLP}} \end{bmatrix})$$

$$\mathbf{h} = \begin{bmatrix} \alpha \mathbf{h}^{\text{GMF}} \\ (1 - \alpha) \mathbf{h}^{\text{MLP}} \end{bmatrix}$$

where  $\mathbf{z}_u^{\text{GMF}}$  and  $\mathbf{z}_u^{\text{MLP}}$  are the outputs of the GMF and MLP components respectively,  $\mathbf{h}$  is the weight vector applying on the outputs of the two components,  $\alpha$  is a hyperparameter controlling

the importance of each component and  $\sigma(\cdot)$  is the non-linear activation function.

**Two-Tower Model** is widely used for large-scale and real-time applications [? ? ]. It features two distinct neural network “towers”: one for encoding user information and the other for encoding item information. The Two-Tower Model framework generally follows NCF with a slight modification, which integrates the neural network components into the embedding layers to form two separate towers but keeping the collaborative filtering layer simple (it simply takes the dot product of embeddings in most cases), in contrast to NCF where neural networks are used in collaborative filtering layer but keeping embedding layer simple. Figure 1.1 visualises the architectural differences between NCF and the Two-Tower Model.

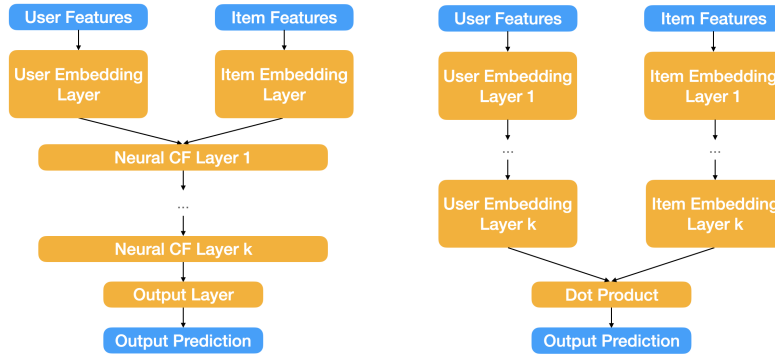


Figure 1.1: High-level diagram of NCF (left) and the Two-Tower Model (right).

The user (item) tower includes an input layer and subsequent embedding layers. The input feature vector can be a one-hot encoded identifier with additional side information. The embedding layers project the input features non-linearly into the latent user (item) space, yielding the user (item) embeddings. The output layer combines the user and item embeddings from the two towers by computing their dot product or another similarity measure. This combined score is then passed through an activation function to produce the final prediction  $\hat{r}_{ui}$ .

The Two-Tower Model allows for efficient real-time retrieval of relevant items by precomputing the embeddings for all items and comparing them to the user embeddings during query time. This model is particularly effective for scenarios where user and item features are high-dimensional and sparse, such as in recommendation systems for e-commerce and streaming services.

### 1.2.3 Graph Representation Learning

There are various approaches for learning from graph data. Traditional methods follow the standard machine learning paradigm by first extracting statistics or features based on heuristic functions or domain knowledge. Modern approaches focus on learning graph representations through node-level, edge-level or graph-level embeddings in the latent space. These crafted or learned features can then serve as inputs for standard machine learning algorithms for downstream tasks.

Machine learning tasks for graph data can be divided into three main categories: supervised, unsupervised and semi-supervised. Supervised tasks include graph classification and regression. Unsupervised tasks involve clustering different graphs and community detection within a graph. Semi-supervised tasks leverage both labeled and unlabeled data for inference, for example node

classification and link prediction [? ]. For general recommendation tasks, it is common to form a bipartite graph based on user-item interactions. This results a semi-supervised link prediction task, where the model should predict a link between the corresponding user and item node pair whenever there is an user-item interaction. Figure 1.2 illustrates a bipartite graph based on the toy movie ratings example in Table 1.1, where historical interactions are connected by edges with the corresponding ratings as weights.

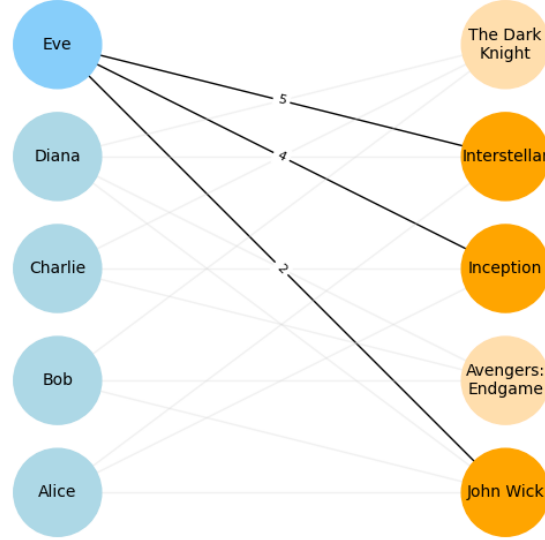


Figure 1.2: Bipartite graph representation of toy movie ratings example in Table 1.1.

Before delving into the details of graph representation learning, we should borrow some formal definitions from graph theory and computer science regarding graphs and their data structures.

**DEFINITION 2** An undirected graph is an ordered pair  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  where  $\mathcal{V} \neq \emptyset$  is a set of vertices and  $\mathcal{E} \subseteq \{\{u, v\} \mid u, v \in \mathcal{V}, u \neq v\}$  is a set of edges, defined as unordered pairs of  $\mathcal{V}$ .

**DEFINITION 3** A directed graph is an ordered pair  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  where  $\mathcal{V} \neq \emptyset$  is a set of vertices and  $\mathcal{E} \subseteq \{\{u, v\} \mid u, v \in \mathcal{V}, u \neq v\}$  is a set of directed edges, defined as ordered pairs of  $\mathcal{V}$ .

**DEFINITION 4** An adjacency list is a data structure used to represent a graph. For each vertex  $v \in \mathcal{V}$ , the adjacency list stores the set of vertices  $\mathcal{U} \subset \mathcal{V}$  adjacent to  $v$ .

**DEFINITION 5** An adjacency matrix is a data structure used to represent a graph. It is a square matrix  $\mathbf{A} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$  where  $A_{uv} = 1$  if  $\{u, v\} \in \mathcal{E}$  else  $A_{uv} = 0$  for any pairs of node  $u, v \in \mathcal{V}$ . If the graph contains weighted edges,  $A_{uv}$  can hold the weight of the corresponding edge. For undirected graphs,  $\mathbf{A}$  is symmetric.

Throughout the thesis, we will be focusing on undirected graphs. Additionally, the adjacency matrix will frequently appear due to its useful properties in the context of mathematical analysis and numerical algorithms.



## Node Features Approaches

In standard machine learning, extracting meaningful features from data is the crucial first step. For graph data, one basic node-level feature is the node degree  $d_u$ , which is the count of number of edges incident to a node  $u \in \mathcal{V}$ . It can be computed as the sum over rows or columns of the adjacency matrix  $\mathbf{A}$ :

$$d_u := \sum_{v \in \mathcal{V}} A_{uv}$$

one can also form the diagonal degree matrix  $\mathbf{D}$  with degree  $d_u$  being the  $u$ -th diagonal element, which is useful in some context.

The node degree counts its number of neighbours, but it does not always adequately reflect node importance within a graph. To more accurately measure a node's significance, we can utilise several metrics collectively known as node centrality measures. One example is the eigenvector centrality, here we state the formula of eigenvector centrality measure  $e_u$  of node  $u$ :

$$e_u = \frac{1}{\lambda} \sum_{v \in \mathcal{V}} A_{uv} e_v$$

By writing  $\mathbf{e} \in \mathbb{R}^{|\mathcal{V}|}$  as a vector of centralities of all nodes, we can easily see this resembles the eigenvector equation of the adjacency matrix  $\mathbf{A}$ :

$$\lambda \mathbf{e} = \mathbf{A} \mathbf{e}$$

By the fact that  $\mathbf{A}$  is square and positive semi-definite, and is irreducible for a connected and undirected graph, we can apply the Perron-Frobenius Theorem to determine that the vector of centrality values  $\mathbf{e}$  is given by the eigenvector corresponding to the largest eigenvalue of  $\mathbf{A}$ . Hence we introduce an iterative algorithm to find the eigenvector centrality measure  $e_u$  of any graph:

$$\begin{aligned} \mathbf{e}^{(0)} &= \begin{bmatrix} 1 & \dots & 1 \end{bmatrix}^\top \\ \mathbf{e}^{(t+1)} &= \text{normalise}(\mathbf{A} \mathbf{e}^{(t)}) \end{aligned}$$

where  $\text{normalise}(\cdot)$  returns a normalised vector summing up to 1, and the converged vector of centralities  $\mathbf{e}^{(\infty)} := \mathbf{e}^*$  is the desired measure. To understand why this algorithm works, we can first imagine  $\mathbf{e}^{(1)}$  contain the degrees of all nodes, i.e. the number of edges (1-step paths) leading to each node. In general,  $\mathbf{e}^{(t)}$  contain the number of  $t$ -step paths leading to each node. The converged numbers are proportional to the number of visits on each node with infinitely long paths [?]. This concept is connected to random walks and the famous PageRank algorithm by Google [?].

While numerous techniques exist for extracting both node-level and graph-level information, we will not go too deep here since our focus is training node embeddings from graphs instead of hand-crafting features.

## Graph Convolution

The main concept revolves around Graph Neural Networks (GNNs) is the graph convolution operation and it remains as the backbone of many state-of-the-art architectures. First of all, we should introduce an important quantity in graph representation learning, the graph Laplacian:

$$\mathbf{L} = \mathbf{D} - \mathbf{A}$$

where  $\mathbf{D}$  and  $\mathbf{A}$  are the degree matrix and adjacency matrix respectively, such that  $\mathbf{L} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$  is also square, symmetric and positive semi-definite. Although the graph Laplacian encodes same information as the adjacency matrix, its unique spectral properties are very important and provide a foundation for extending the standard convolution operation to the graph domains.

Note that there are popular variants of normalised Laplacian such as the symmetric normalised Laplacian  $\mathbf{L}_{sym} = \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2}$  and the random walk Laplacian  $\mathbf{L}_{RW} = \mathbf{D}^{-1} \mathbf{L}$  which exhibit similar algebraic properties but differ by a normalisation constant. The choices between these Laplacian variants often depend on the specific requirements of the model and the nature of the problem being addressed [? ? ].

Note that the spectral decomposition of  $\mathbf{L}$  is the following:

$$\begin{aligned} \mathbf{L} &= \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top \\ \mathbf{U} &= \begin{bmatrix} \mathbf{u}_0 & \mathbf{u}_1 & \cdots & \mathbf{u}_{|\mathcal{V}|-1} \end{bmatrix} \\ \mathbf{\Lambda} &= \begin{bmatrix} \lambda_0 & 0 & \cdots & 0 \\ 0 & \lambda_1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_{|\mathcal{V}|-1} \end{bmatrix} \\ 0 &= \lambda_0 \leq \lambda_1 \leq \cdots \leq \lambda_{|\mathcal{V}|-1} \end{aligned}$$

where  $\mathbf{U}$  contains the complete set of orthonormal eigenvectors and  $\mathbf{\Lambda}$  is a diagonal matrix of ordered real non-negative eigenvalues with respect to the eigenvectors.

This is analogous to the Fourier transform in traditional signal processing where  $\mathbf{U}$  is the graph Fourier modes (basis) and  $\mathbf{\Lambda}$  is the frequencies. Thus the graph Fourier Transform of a signal  $x \in \mathbb{R}^{|\mathcal{V}|}$  can be defined as  $\hat{x} = \mathbf{U}^\top x$  with the inverse transform  $x = \mathbf{U} \hat{x}$ . Then, by the Convolution Theorem, graph convolution of two signals  $x *_G y$  can be defined as the Hadamard product in the Fourier domain:  $x *_G y = \mathbf{U}((\mathbf{U}^\top x) \odot (\mathbf{U}^\top y))$ .

We can also perform the same convolution mechanism with some filter  $g_\theta$  such that original signal  $x$  is filtered as,

$$\begin{aligned} \hat{x} &= x *_G g_\theta \\ &= \mathbf{U}((\mathbf{U}^\top x) \odot (\mathbf{U}^\top g_\theta)) \end{aligned}$$

A natural choice is to learn a non-parametric filter  $\theta := \mathbf{U}^\top g_\theta \in \mathbb{R}^{|\mathcal{V}|}$  in the Fourier space, where  $\theta$  is optimised on data. With such filter, the convolution operation can be rewritten without

Hadamard product,

$$\begin{aligned}\hat{x} &= \mathbf{U}((\mathbf{U}^\top x) \odot (\mathbf{U}^\top g_\theta)) \\ &= \mathbf{U}((\mathbf{U}^\top x) \odot \theta) \\ &= (\mathbf{U} \text{diag}(\theta) \mathbf{U}^\top) x\end{aligned}$$

However, such non-parametric filter has no real dependency on the graph structure and it is not localised. Also, its learning complexity is  $\mathcal{O}(|\mathcal{V}|)$  which scales with graph sizes [? ? ].

To make graph convolution meaningful and computationally efficient, especially for large-scale graphs, the spectral filters are often approximated using polynomials of the Laplacian. A polynomial filter of degree  $k$  can be defined as  $\sum_{i=0}^k \theta_i \mathbf{\Lambda}^i$ , where  $\theta_i$  is the polynomial coefficient. In this case the filter is  $k$ -localised and its complexity  $\mathcal{O}(k)$  which equals to the “size” of filter, these two properties closely resembles the filters of Convolutional Neural Networks (CNNs) [? ? ].

The graph convolution operation on signal  $x$  using a polynomial filter can be expanded as:

$$\begin{aligned}\hat{x} &= (\mathbf{U} \sum_{i=0}^k \theta_i \mathbf{\Lambda}^i \mathbf{U}^\top) x \\ &= \sum_{i=0}^k \theta_i \mathbf{U} \mathbf{\Lambda}^i \mathbf{U}^\top x \\ &= \sum_{i=0}^k \theta_i \mathbf{L}^i x =: p_\theta(\mathbf{L}) x\end{aligned}$$

We can show that the spectral graph convolution with a polynomial filter can be equivalently expressed as a polynomial of the Laplacian matrix  $p_\theta(\mathbf{L})$  on signal  $x$ . In real-world scenarios where graphs are very large but sparse that number of vertices  $|\mathcal{V}| \gg |\mathcal{E}|$ , using  $p_\theta(\mathbf{L})$  to apply graph convolution is much more efficient than first computing eigen-decomposition of  $\mathbf{L}$  and follow the general steps. Since the former method has complexity  $\mathcal{O}(k|\mathcal{E}|)$  much smaller than  $\mathcal{O}(|\mathcal{V}|^2)$  [? ].

To further understand graph convolution, we can examine the effect of coefficients  $\theta_i$  in the polynomial filter  $p_\theta(\mathbf{L})$ . Expanding a degree  $k$  polynomial of Laplacian:

$$\begin{aligned}\hat{x} &= p_\theta(\mathbf{L}) x \\ &= \sum_{i=0}^k \theta_i \mathbf{L}^i x \\ &= (\theta_0 \mathbf{I} + \theta_1 \mathbf{L} + \dots + \theta_k \mathbf{L}^k) x\end{aligned}$$

We first consider the case where  $\theta_0 = 1$  and all other  $\theta_i = 0$ , we are left with a simple identity transformation  $\hat{x} = \mathbf{I}x$ . This can be interpreted as a skip connection in the context of neural networks, retaining the original signal of each node.

Then, consider the case where  $\theta_1 = 1$  and all other  $\theta_i = 0$ , we have a linear transformation  $\hat{x} = \mathbf{L}x = (\mathbf{D} - \mathbf{A})x$ . Recall  $\mathbf{D}$  is the degree matrix encoding the node itself and  $\mathbf{A}$  is the adjacency matrix encoding the nodes’ neighbourhood. Therefore,  $\mathbf{L}x$  computes the signal differences between nodes and their neighbourhoods, it captures the 1-hop neighbour signal variations. For higher

degrees  $k \geq 1$ ,  $\mathbf{L}^k$  captures the signal variations until the  $k$ -hop neighbourhood, this demonstrates the  $k$ -localised property of the polynomial filter and the foundations behind graph convolution.

The graph convolution operation can be seen as “message passing” between the local neighbourhood, the general message passing framework consists of two steps, aggregation and update:

$$\begin{aligned} m_i &= \text{AGGREGATE}(\{h_j : j \in N(i)\}) \\ h'_i &= \text{UPDATE}(h_i, m_i) \end{aligned}$$

where  $h_i$  and  $h'_i$  is the initial and updated  $i$ -th node’s representation. In the aggregation step, each node collects messages (node representations) from its neighbours. The aggregation function can vary, but common choices include sum, mean or max operations. In the update step, each node updates its representation based on the aggregated message and its current state [? ].

The message passing concept is fundamental to GCNs and is highly relevant to NGCF and LightGCN. To qualitatively understand how GCN works, we can imagine an over-parameterised aggregator being the neural network. It tranverses through the graph, collects and aggregates each node’s neighbours represetaions and use its output to update the representation of center node. Figure 1.3 provides a visual aid of a GCN tranversing the graph and perform message passing on each node, note that the order of graph traversal does not matter in practice.

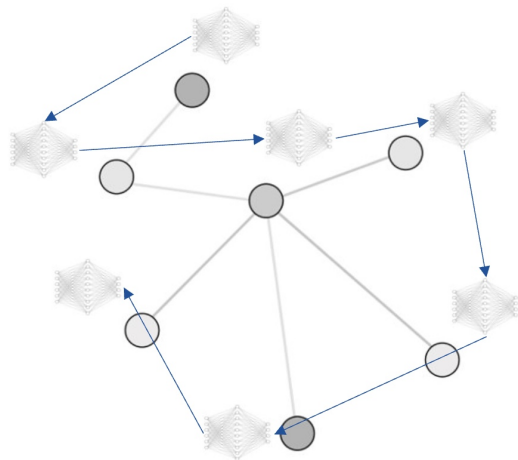


Figure 1.3: Visual aid of GCN traversing a graph for message passing, figure materials sourced from Daigavane et al. [? ]

## 1.3 Related Work

### 1.3.1 Graph Representation Learning and Spectral Convolutions

Graph representation learning and Graph Neural Networks (GNNs) have seen significant advancements in recent years. These approaches aim to learn low-dimensional embeddings of nodes, edges or entire graphs that capture structural and feature information.

One of the foundational works in this area is by Bruna et al. [? ] who first proposed the idea of generalising Convolutional Neural Networks (CNNs) to graph-structured data. They introduced

spectral graph convolutions, defining graph convolution operations based on the eigenvectors of the graph Laplacian. This spectral approach laid the groundwork for many subsequent developments in GNNs.

Building on this spectral perspective, Defferrard et al. [?] proposed ChebNet, which uses Chebyshev polynomials to approximate spectral graph convolutions. This approach allows for localised filters and reduces the computational complexity compared to earlier spectral methods. ChebNet demonstrated how graph convolutions could be efficiently implemented and applied to various graph-based learning tasks.

Kipf and Welling [?] further simplified the spectral approach with their Graph Convolutional Network (GCN) model. By using a first-order approximation of ChebNet, they created a simple and effective graph convolutional layer that aggregates information from a node’s immediate neighbours. GCN has become one of the most widely used and influential models in the field, serving as a baseline for many subsequent works.

From a theoretical perspective, Xu et al. [?] provided important insights into the expressive power of GNNs. They showed that message-passing neural networks are at most as powerful as the Weisfeiler-Lehman graph isomorphism test in terms of distinguishing graph structures. This work highlighted both the capabilities and limitations of current GNN architectures.

These works collectively established the theoretical foundations and practical implementations of graph convolutions, paving the way for more advanced and task-specific GNN architectures.

### 1.3.2 Graph-Based Recommendation Models

Recent advancements in graph-based models for recommendation systems have centered on leveraging local connectivity information to enhance performance, notable examples include Collaborative Memory Network (CMN), Graph Convolutional Matrix Completion (GC-MC) and High-Order Proximity for implicit Recommendation (HOP-Rec).

CMN introduces a memory component and a neural attention mechanism to capture complex user-item interactions. CMN effectively combines the strengths of latent factor models and neighbourhood-based methods in a non-linear fashion. The memory component encodes rich feature representations, while the neural attention mechanism infers user-specific contributions from the broader community [?].

GC-MC approaches the matrix completion problem as a link prediction task on a bipartite user-item graph. GC-MC employs graph convolution layers to learn user and item embeddings by propagating information across the graph structure. By using relation-specific weight matrices in the graph convolution operations, GC-MC distinguishes between different types of user-item interactions [?].

HOP-Rec introduces a novel approach to implicit recommendation by integrating latent factor models with graph-based techniques. It leverages high-order proximity information within the user-item interaction graph to enhance recommendation accuracy. HOP-Rec captures both direct and indirect user-item connections through multiple hops in the graph [?].

These graph-based models demonstrate the efficacy of incorporating graph structures and local connectivity information into recommendation systems. However, these architectures are often highly specialised with custom-engineered components, making them challenging to extend and

further improve.

### 1.3.3 GNN-Based Recommendation Models

There have been several efforts to incorporate the general GNN frameworks into recommendation systems. One notable example is PinSage [? ], which extends the foundational work laid out by the GraphSAGE framework [? ].

GraphSAGE introduced an inductive approach for generating node embeddings in large graphs. Unlike traditional methods that require retraining the model when new nodes are added, GraphSAGE can generate embeddings for previously unseen nodes by sampling and aggregating features from a node’s local neighbourhood. This sampling-based method enables the model to scale effectively to large graphs and handle dynamic graph structures [? ]. The flexibility and efficiency of GraphSAGE have made it valuable for subsequent developments in graph-based learning.

Building on the principles of GraphSAGE, PinSage was specifically designed to meet the demands of large-scale recommendation systems. PinSage adapts the neighbourhood sampling strategy of GraphSAGE by using random walks to explore a node’s local neighbourhood, effectively capturing higher-order connectivity in the graph. This approach allows PinSage to learn more meaningful and informative embeddings for nodes (representing users and items) in massive graphs. Additionally, PinSage incorporates a novel pooling mechanism that assigns importance weights to neighbouring nodes based on random walk scores, helping the model to focus on the most relevant connections [? ]. This combination of techniques allows PinSage to scale efficiently to industrial-level recommendation systems, handling billions of nodes and edges.

GNN-based recommendation models are easier to analyse and extend as they share a common framework. This consistency facilitates rapid development, enabling the invention of new message-passing mechanisms and various GCNs architectures.

### 1.3.4 State-of-the-Art GNN-Based Recommendation Models

Our study focuses on Neural Graph Collaborative Filtering (NGCF) and Light Graph Convolution Network (LightGCN) as the representatives of recent GNN-based recommendation models. They have relatively simpler model formations than the previous models yet obtaining similar or even superior performance in many benchmark datasets.

NGCF introduces a novel recommendation framework that explicitly incorporates the user-item interaction graph structure into the embedding learning process. The model propagates embeddings across the user-item interaction graph, refining representations by injecting higher-order structural information, which results in substantial improvements over traditional collaborative filtering methods and other graph-based models [? ].

Building upon the success of NGCF, LightGCN simplifies the design of GCNs for collaborative filtering. It focuses solely on neighbourhood aggregation while removes feature transformation and non-linear activation, two common operations in GCNs, arguing that these are unnecessary for recommendation tasks and may even hinder performance. This simplification not only makes the model easier to train but also leads to improved performance compared to NGCF and other state-of-the-art methods [? ].

These approaches have set the stage for further exploration in graph-based recommendation systems, motivating the development of more advanced GNN architectures and their application to diverse recommendation tasks.

## 1.4 Problem Statement

While traditional collaborative filtering methods could be effective, they often struggle to capture the full complexity of user-item interactions in large-scale recommendation systems. These methods typically rely on straightforward embeddings derived from direct user-item interactions, which may miss the richer collaborative signals embedded in the broader interaction network.

The emergence of graph-based and GNN-based models presents promising solutions by leveraging the structural information within user-item interaction graphs. Although many research has shown success in real-world applications, there remains substantial room for investigation in applying these models directly to real large-scale industry datasets.

There are still many challenges to be addressed, such as the architecture design, model scalability and practical implementation. This project aims to bridge the gap between theoretical advancements in GNN-based recommendation models and their practical application to industry datasets.

## 1.5 Research Objectives

The primary objective of this research is to explore the effectiveness and scalability of GNN-based recommendation models for real-world retail datasets. Specifically, this study aims to:

- Implement and evaluate the performance of the target GNN-based recommendation models, including NGCF, LightGCN and their GraphSAGE variants, in comparison to traditional matrix factorisation baselines across datasets of different scales.
- Examine the quality of embeddings and the resulting recommendations and justify the impact of incorporating high-order connectivity information in the embedding process
- Assess the scalability and computational efficiency of GNN-based recommendation models when applied to large-scale retail datasets, identifying potential bottlenecks and proposing strategies for optimisation.
- Understand the trade-offs between model complexity, performance, and computational requirements on large-scale datasets.

## 1.6 Significance of the Study

This research has significant implications for both academic and industrial applications of recommendation systems.

For the retail industry, it provides valuable insights into the potential of GNN-based models to improve recommendation accuracy, which could lead to enhanced downstream tasks performance,

increased sales and greater user engagement. From an academic standpoint, the study contributes to the understanding of how theoretical advancements in GNN-based recommendation models can be effectively applied to real-world large-scale datasets.

Additionally, the focus on scalability and efficiency provides practical guidance for implementing advanced models in practical resource-constrained industrial settings. By comparing different architectures and up-scaling techniques, this research can inform future model development and optimisation efforts within the field of recommender systems, influencing the design of next-generation recommendation engines.



# Chapter 2

## Methodology

### 2.1 Datasets

#### 2.1.1 Data Sources

This study draws upon two primary data sources: the Tesco purchase history and supplementary product information.

The primary data source is the Tesco purchase history, which encompasses the records of transactions over a one-year period. This dataset comprises anonymised triplets, each containing a unique customer identifier, a product identifier and the quantity purchased. To ensure data privacy and confidentiality, all customer and product information is anonymised, with each entity represented by a distinct identifier. This transactional data undergoes preprocessing and filtering to construct user-item interaction bipartite graphs of varying scales for training Graph Neural Network (GNN) models.

The secondary dataset containing detailed product information for complementing the purchase history data. This dataset links each product identifier to its name and classifications within Tesco commercial hierarchy, which ranges from level 1 representing the broadest category, to level 5 representing the most granular category. While not directly incorporated into the core GNN training process, this supplementary product data is used for miscellaneous experiments and analyses, such as embedding clustering analysis, visualisation of embeddings and modelling with external features.

#### 2.1.2 Data Preprocessing

Both datasets are preprocessed before being applied in experiments and analyses. The purchase history data requires more sophisticated data processing and manipulation techniques, while the product data goes through trivial processing such as removing duplicates and dropping unrelated entries. Below we outline the approach for processing purchase history data:

## Outliers Removal

The transactional data sometimes contains outliers where they buy a unrealistically large amount of products compared to most of the customers. They might be bulk purchasers who source from Tesco for operating other companies or restaurants, or simply artifacts from software testing or bugs. In either way this kind of outliers may do not represent the purchase behaviour of the majority and should be removed to ensure fair sampling and message propagation during graph inference. We will remove any user who purchased more than 10% of the product base for the Tesco purchase history data.

## Graph Pruning

The bipartite graph built from raw transactional data is typically large but extremely sparse, primarily due to the prevalence of inactive users who make infrequent purchases. The performance of recommendation models on these users may be unsatisfactory and difficult to evaluate in offline settings. Moreover, an unnecessarily large graph increases the computational burden during model training and inference.

Our study mainly employs  $k$ -core filtering to ensure every node in the filtered graph contains at least  $k$  neighbours, effectively removing inactive users and unpopular items. We implement this process with separate core numbers for user and item nodes, allowing for finer control over the filtering process especially when the user and item nodes are exhibiting different patterns. Algorithm 1 shows the general logic of  $k$ -core filtering.

---

**Algorithm 1:** K-core filtering for bipartite graphs.

---

**Input:** Bipartite graph  $\mathcal{G}(\mathcal{U}, \mathcal{I}, \mathcal{E})$  where  $\mathcal{U}$  is the set of users,  $\mathcal{I}$  is the set of items, and  $\mathcal{E}$  is the set of edges; core number for users  $k_{\text{user}}$ ; core number for items  $k_{\text{item}}$

**Output:** K-core subgraph  $\mathcal{G}_k(\mathcal{U}_k, \mathcal{I}_k, \mathcal{E}_k)$

```
1  $\mathcal{U}_k \leftarrow \mathcal{U}, \mathcal{I}_k \leftarrow \mathcal{I}, \mathcal{E}_k \leftarrow \mathcal{E}$ 
2  $converged \leftarrow False$ 
3 while  $\neg converged$  do
4    $converged \leftarrow True$ 
5   for  $u \in \mathcal{U}_k$  do
6     if  $degree(u) < k_{user}$  then
7        $\mathcal{U}_k \leftarrow \mathcal{U}_k \setminus \{u\}$ 
8        $\mathcal{E}_k \leftarrow \mathcal{E}_k \setminus \{(u, i) | i \in \mathcal{I}_k\}$ 
9        $converged \leftarrow False$ 
10    end
11  end
12  for  $i \in \mathcal{I}_k$  do
13    if  $degree(i) < k_{item}$  then
14       $\mathcal{I}_k \leftarrow \mathcal{I}_k \setminus \{i\}$ 
15       $\mathcal{E}_k \leftarrow \mathcal{E}_k \setminus \{(u, i) | u \in \mathcal{U}_k\}$ 
16       $converged \leftarrow False$ 
17    end
18  end
19 end
20 return  $\mathcal{G}_k(\mathcal{U}_k, \mathcal{I}_k, \mathcal{E}_k)$ 
```

---

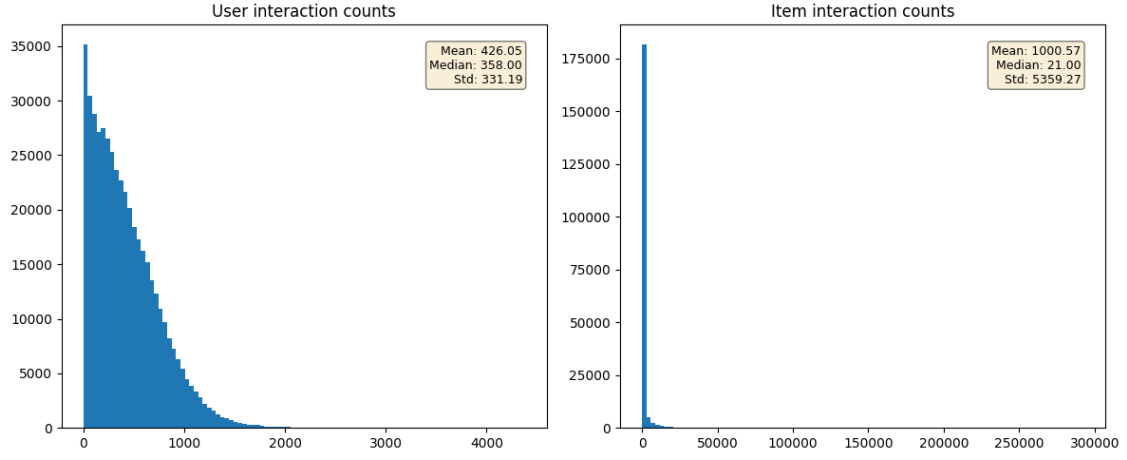
This algorithm iteratively removes user nodes with fewer than  $k_{\text{user}}$  neighbours and item nodes with fewer than  $k_{\text{item}}$  neighbours. The process continues until no further nodes can be removed, returning a denser graph where each user node has a minimum degree of  $k_{\text{user}}$  and each item node has a minimum degree of  $k_{\text{item}}$ .

Note that we use different  $k$ -values for user nodes  $k_{\text{user}}$  and item nodes  $k_{\text{item}}$  since the degree distributions of these two types of nodes can be significantly different. The effects of controlling these different core numbers are demonstrated in Figure 2.1:

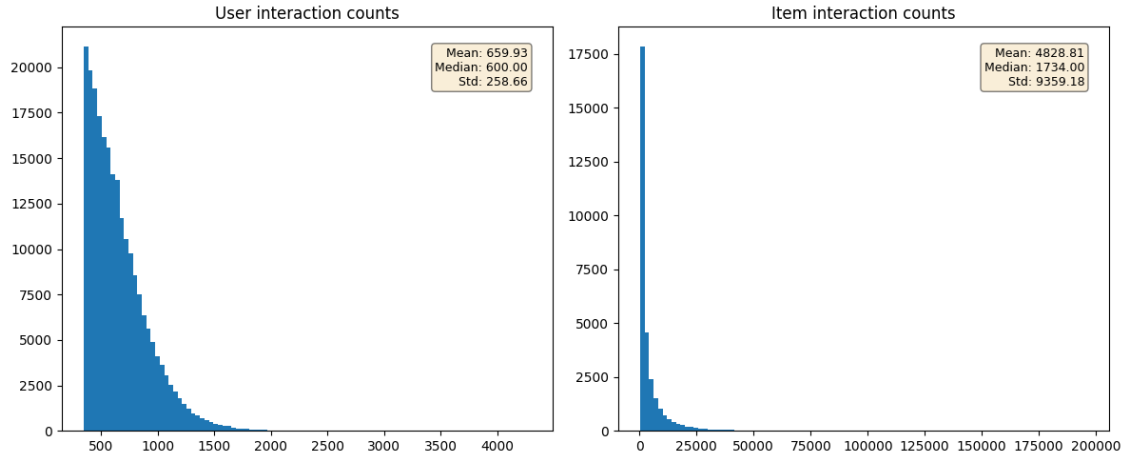
(a) The raw, unfiltered data exhibits positively skewed distributions for both users and items, characterised by long right tails. The user distribution shows a smoother decline in density as degree increases, with relatively close mean and median values. In contrast, the item distribution is notably discontinuous, with most items having degrees in the order of  $10^2$ , while a few rare items reach degrees as high as  $10^4$  and  $10^5$ . This disparity is reflected in the significant difference between the mean and median values for items.

(b) Applying a constant  $k_{\text{user}} = k_{\text{item}} = 350$  for both user and item nodes results in the removal of approximately 50% of users and over 80% of items. This filtering effectively eliminates most unpopular items, reducing the discontinuity in the item distribution. While this approach may be suitable for user filtering, it appears overly stringent for items.

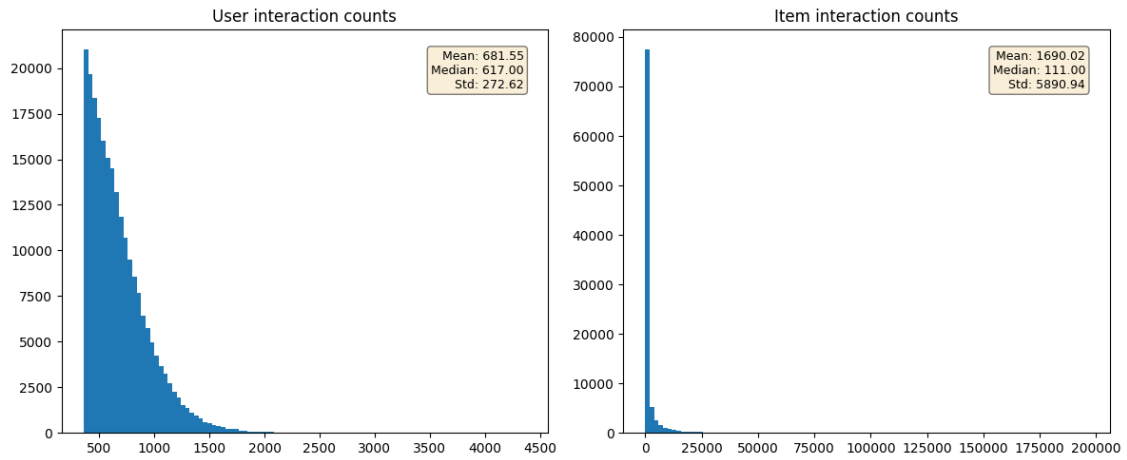
(c) Using the distribution medians as thresholds ( $k_{\text{user}} = 358, k_{\text{item}} = 21$ ) leads to the re-



(a) Total users: 457206 Total items: 194682 Density: 0.00219



(b) Total users: 227584 Total items: 31103 Density: 0.02122



(c) Total users: 228306 Total items: 92083 Density: 0.00740

Figure 2.1: Node degree distributions for different k-core filtering settings. (a) Raw data; (b)  $k_{\text{user}} = 350$ ,  $k_{\text{item}} = 350$  filtered data; (c)  $k_{\text{user}} = 358$ ,  $k_{\text{item}} = 21$  filtered data.

removal of about 50% of both users and items, as expected. The item distribution retains some discontinuity due to the lower threshold, allowing more items to be kept in the filtered graph.

These results suggest that using distribution quantiles to select  $k$ -values is an effective strategy for controlling the proportion of low-density areas to be trimmed from the graph. This approach allows for targeted down-scaling of the graph and increases density. For less aggressive filtering, smaller core numbers (e.g., 10 or 100) can suffice to remove rare, potentially uninteresting nodes without drastically altering the graph structure. The choice of  $k$ -values should depend on the requirements of the recommendation task, balancing between computational efficiency and the preservation of potentially valuable long-tail interactions.

There are also other graph pruning methods that do not significantly reduce graph sizes but decrease the density of the graph, such as edge weight threshold to remove edges with weights less than a certain threshold, edges downsampling to drop edges randomly when the node degree is over a certain threshold. We do not apply these approaches here since we follow the original work in NGCF and LightGCN, also we want denser graphs other than sparser graphs.

## Data Structures

Efficient data representation is crucial for handling large-scale graphs in recommender systems. We utilise several data structures, each offering distinct advantages for different phases of our recommendation pipeline, striking a balance between computational efficiency, memory utilisation and algorithmic requirements.

Edge lists represent the primal form of our raw data, each entry in an edge list contains the user identifier, item identifier and the corresponding purchase quantity. This representation is intuitive and flexible, making it particularly useful during preprocessing. As edge lists can be memory-intensive, we build and process the edge lists in chunks and store them in Parquet format, offering efficient compression and column-based access.

Adjacency lists offer a more compressed representation compared to edge lists, where we maintain a list of neighbours for each node. This structure is more memory-efficient than edge lists for sparse graphs and facilitates efficient node-centric operations and neighbourhood sampling. We leverage adjacency lists primarily for sampling user nodes, positive and negative items during the training phase, where rapid access to node neighbourhoods is crucial.

Adjacency matrices and graph Laplacians encode the entire graph structure in a single large matrix, as the scale of our graphs is very large. We utilise the sparse implementations for matrices and tensors as well as the respective sparse matrices and tensors operations provided by SciPy and PyTorch for memory efficiency and computational speed. This approach is particularly beneficial for our graph neural network models, enabling fast parallelised message propagation and gradient computations during training.

### 2.1.3 Experimental Datasets

#### Yelp2018

The Yelp-2018 dataset is a standard benchmark for evaluating recommendation models, and is used in the original papers of Neural Graph Collaborative Filtering (NGCF) and Light Graph Convolution Network (LightGCN) [? ? ]. Originating from the 2018 Yelp Challenge, it contains

real-world data on local businesses, user reviews and user-business interactions. We apply the same 10-core filtering and 80-20 train-test split with 10% of train data being validation data as stated in the original papers to maintain consistency. This dataset validates our model implementations by verifying the replication of NGCF and LightGCN results before moving on to the proprietary Tesco datasets.

### Tesco-5k Dataset

The Tesco-5k dataset consists of purchase records from the 5,000 most active users within a development subset of Tesco full purchase history data. This dataset undergoes basic preprocessing steps including outlier removal and 10-core filtering, we split the data into train, validation and test sets evenly to facilitate metric comparison across subsets. Its small size allows for quick training and evaluation, making it ideal for hyperparameter tuning before scaling to larger datasets.

### Tesco-20k Dataset

The Tesco-20k dataset is an expanded version of Tesco-5k, encompassing purchase data from the top 20,000 active users. It undergoes the same preprocessing and even data split strategy. This mid-sized dataset is used to train and evaluate models with the optimal parameters identified in the Tesco-5k experiments. Since its scale is comparable to common benchmark datasets, it provides a reliable platform for evaluating ranking metrics, embedding analysis and exploring downstream applications, while ensuring the robustness and generalisation of the models.

### Tesco-228k Dataset

The Tesco-228k dataset offers a comprehensive view of customer behavior, representing a substantial portion of Tesco’s full purchase data. Due to its large size and sparsity, we apply a more sophisticated (358, 21)-core filtering based on median node distribution values, along with the same outlier removal and even data split strategy as in previous datasets. This large-scale dataset is for assessing model scalability and convergence using the optimised parameters from earlier experiments. It ensures that the models generalise effectively across various data scales, validating the findings from smaller datasets in more extensive, real-world scenarios.

Table 2.1 shows the summarised statistics of the dataset being used in our studies, we will detail each dataset in the following.

Table 2.1: Original and reproduced test set performance of NGCF and LightGCN.

| Dataset           | User nodes | Item nodes | Edges       | Density |
|-------------------|------------|------------|-------------|---------|
| <b>Yelp2018</b>   | 31,668     | 38,048     | 1,561,406   | 0.0013  |
| <b>Tesco-5k</b>   | 5,000      | 11,524     | 716,333     | 0.0124  |
| <b>Tesco-20k</b>  | 20,000     | 18,691     | 2,494,938   | 0.0067  |
| <b>Tesco-228k</b> | 228,306    | 92,083     | 155,622,188 | 0.0074  |

## 2.2 Model Architectures

This study mainly focuses on the NGCF and LightGCN frameworks which leverage GNNs specifically GCNs into modelling user-item interactions from the bipartite graph structure. These approaches learn user and item graph embeddings through high-order connectivity information from the bipartite graph.

This study also explores potential solutions of NGCF and LightGCN for large-scale training. Inspired by GraphSAGE, the core neighbour sampling and aggregation algorithm can be integrated into the message propagation layers of NGCF and LightGCN, serving as the aggregator. The integration significantly reduces GPU memory footprint during training on large graphs, as message propagation is performed on sampled subgraphs, which is essential for scaling our models to real-world datasets with large number of users and items.

### 2.2.1 Neural Graph Collaborative Filtering (NGCF)

The NGCF framework performs graph convolution and collaborative filtering through multi-hop neighbours. The model consists of three primary components: the embedding layer, the embedding propagation layers and the prediction layer [? ].

#### Embedding Layer

The embedding layer first randomly initialises trainable embeddings for all users and items. Each user  $u$  and item  $i$  is described with an embedding vector  $\mathbf{e}_u \in \mathbb{R}^d$  and  $\mathbf{e}_i \in \mathbb{R}^d$  respectively where  $d$  is the embedding size. To facilitates efficient lookup and computation, these embeddings form a matrix:

$$\mathbf{E} = \begin{bmatrix} \mathbf{e}_{u_1} & \cdots & \mathbf{e}_{u_N} & \mathbf{e}_{i_1} & \cdots & \mathbf{e}_{i_M} \end{bmatrix}^\top$$

where  $N$  is the total number of users and  $M$  is the total number of items.

#### Embedding Propagation Layers

The embedding propagation layer refines the input embeddings by collecting its neighbours' embeddings and capturing collaborative signal along the bipartite graph structure. The propagation layer consists of two steps: message construction and message aggregation.

During message construction, message  $m_{u \leftarrow i}$  from a neighbour item  $i$  to user  $u$  is constructed:

$$\mathbf{m}_{u \leftarrow i} = \frac{1}{\sqrt{|N_u||N_i|}} (\mathbf{W}_1 \mathbf{e}_i + \mathbf{W}_2 (\mathbf{e}_i \odot \mathbf{e}_u))$$

where  $\mathbf{W}_1$  and  $\mathbf{W}_2$  are trainable weight matrices,  $\odot$  denotes element-wise product. The message is a normalised sum of two signals: The first term corresponds to the conventional graph convolution operation, the second term corresponds to the collaborative filtering signal between the item-user pair. The sum is then normalised by  $p_{ui} := \frac{1}{\sqrt{|N_u||N_i|}}$  which controls the decay factor on the propagation edge  $(u, i)$ , this ensures the message to decay as the path length increases.

Self-connected message is also constructed  $\mathbf{m}_{u \leftarrow u} = \mathbf{W}_1 \mathbf{e}_u$  using the same weight matrix  $\mathbf{W}_1$ , this retains the original information of the node and acts like a skip connection.

During message aggregation, the self-connected message of  $u$  and messages from  $u$ 's neighbourhood are aggregated to refine  $u$ 's embeddings:

$$\mathbf{e}'_u = \text{LeakyReLU} \left( \mathbf{m}_{u \leftarrow u} + \sum_{i \in N_u} \mathbf{m}_{u \leftarrow i} \right)$$

where  $\mathbf{e}'_u$  is the refined embeddings of  $u$  after a pass of embedding propagation layer.

Likewise, we can compute the refined embeddings  $\mathbf{e}'_i$  of item  $i$  by propagating messages from its neighbouring users.

For high-order message propagation, we can simply stack multiple embedding propagation layers which iteratively refines the embeddings output of the previous layer. In general, embeddings at layer  $k$  can be computed using embeddings at layer  $k - 1$  in a recursive manner as below:

$$\begin{aligned} \mathbf{m}_{u \leftarrow i}^{(k)} &= p_{ui} \left( \mathbf{W}_1^{(k)} \mathbf{e}_i^{(k-1)} + \mathbf{W}_2^{(k)} (\mathbf{e}_i^{(k-1)} \odot \mathbf{e}_u^{(k-1)}) \right) \\ \mathbf{m}_{u \leftarrow u}^{(k)} &= \mathbf{W}_1^{(k)} \mathbf{e}_u^{(k-1)} \\ \mathbf{e}_u^{(k)} &= \text{LeakyReLU} \left( \mathbf{m}_{u \leftarrow u}^{(k)} + \sum_{i \in N_u} \mathbf{m}_{u \leftarrow i}^{(k)} \right) \end{aligned}$$

In practice, message propagation is implemented in matrix form, allowing for simultaneous updates of all nodes and enabling efficient computation on GPUs. Additionally, the practical implementation slightly differs from the theoretical model above by incorporating collaborative signals into self-connected messages, which enhances the representation power. The updated embeddings at layer  $k$  are given by:

$$\mathbf{E}^{(k)} = \text{LeakyReLU} \left( \mathbf{L}_{\text{norm}} \mathbf{E}^{(k-1)} \mathbf{W}_1^{(k)} + \mathbf{L}_{\text{norm}} \mathbf{E}^{(k-1)} \odot \mathbf{E}^{(k-1)} \mathbf{W}_2^{(k)} \right)$$

where  $\mathbf{E}^{(k)} \in \mathbb{R}^{(N+M) \times d_k}$  is the embeddings matrix at layer  $k$  with  $\mathbf{E}^{(0)}$  being the initial embeddings lookup from the embedding layer.  $\mathbf{L}_{\text{norm}} \in \mathbb{R}^{(N+M) \times (N+M)}$  is the normalised Laplacian matrix with self-connection which is commonly used in GNNs.  $\mathbf{A} \in \mathbb{R}^{(N+M) \times (N+M)}$  is the adjacency matrix and  $\mathbf{R} \in \mathbb{R}^{N \times M}$  is the user-item interaction matrix.

$$\mathbf{L}_{\text{norm}} = \mathbf{D}^{-\frac{1}{2}} (\mathbf{A} + \mathbf{I}) \mathbf{D}^{-\frac{1}{2}} \text{ and } \mathbf{A} = \begin{bmatrix} \mathbf{0} & \mathbf{R} \\ \mathbf{R}^\top & \mathbf{0} \end{bmatrix}$$

## Prediction Layer

The prediction layer gathers embeddings from different layers and output the concatenated embeddings as the final representation. For any user  $u$  and item  $i$ , the final embeddings output  $\mathbf{e}_u^*$ ,  $\mathbf{e}_i^*$  of a  $K$ -layer model is:

$$\begin{aligned} \mathbf{e}_u^* &= \mathbf{e}_u^{(0)} \parallel \dots \parallel \mathbf{e}_u^{(K)} \\ \mathbf{e}_i^* &= \mathbf{e}_i^{(0)} \parallel \dots \parallel \mathbf{e}_i^{(K)} \end{aligned}$$



where  $\parallel$  denotes concatenation. The affinity score for any  $(u, i)$  pair is the dot product of their final embeddings  $\hat{y}_{\text{NGCF}}(u, i) = \mathbf{e}_u^* \cdot \mathbf{e}_i^*$ .

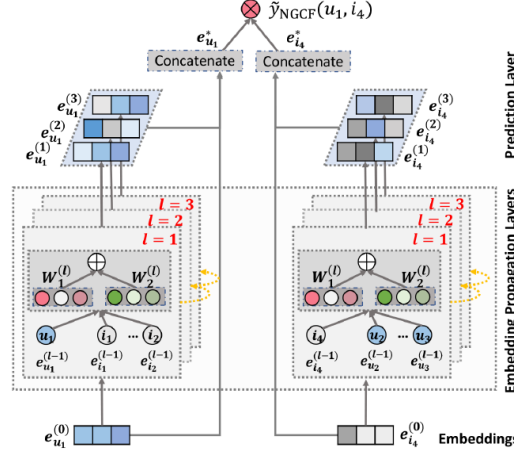


Figure 2.2: NGCF architecture overview sourced from the original paper [? ].

### 2.2.2 Light Graph Convolution Networks (LightGCN)

The LightGCN framework is a more lightweight alternative to its predecessor NGCF, while offering superior performance as demonstrated in several research papers. The model solely focuses on graph convolution and removes feature transformation matrices as well as non-linearity, making it much more compact and parameter efficient [? ].

#### Embedding Layer

Same as NGCF, each user  $u$  and item  $i$  is described with an trainable embedding vector  $\mathbf{e}_u$  and  $\mathbf{e}_i$  respectively. These embeddings form a matrix  $\mathbf{E}$  for efficient lookup and computation.

#### Embedding Propagation Layers

The embedding propagation layer performs the “Light Graph Convolution” operation without additional trainable parameters and non-linearity, it outputs the layer  $k$  embeddings of any user or item node:

$$\begin{aligned}\mathbf{e}_u^{(k)} &= \sum_{i \in \mathcal{N}_u} p_{ui} \mathbf{e}_i^{(k-1)} \\ \mathbf{e}_i^{(k)} &= \sum_{u \in \mathcal{N}_i} p_{ui} \mathbf{e}_u^{(k-1)}\end{aligned}$$

where  $p_{ui} = \frac{1}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_i|}}$  is the decay factor inversely proportional to the number of neighbours of both nodes. This layer refines the node’s embeddings with a normalised sum of its neighbours’ embeddings.

The matrix form of “Light Graph Convolution” is the product of the normalised Laplacian without self-connection  $\mathcal{L}_{\text{norm}}$  and the embedding matrix  $\mathbf{E}$ :

$$\begin{aligned}\mathbf{E}^{(k)} &= (\mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}) \mathbf{E}^{(k-1)} \\ &= \mathcal{L}_{\text{norm}} \mathbf{E}^{(k-1)}\end{aligned}$$

### Prediction Layer

The prediction layer gathers embeddings from different layers and takes their element-wise weighted average as the final representation. In terms of the embedding matrix, the final embeddings output  $\mathbf{E}^*$  of a K-layer model is:

$$\begin{aligned}\mathbf{E}^* &= \alpha_0 \mathbf{E}^{(0)} + \alpha_1 \mathbf{E}^{(1)} + \alpha_2 \mathbf{E}^{(2)} + \dots + \alpha_K \mathbf{E}^{(K)} \\ &= (\alpha_0 + \alpha_1 \mathcal{L}_{\text{norm}} + \alpha_2 \mathcal{L}_{\text{norm}}^2 + \dots + \alpha_K \mathcal{L}_{\text{norm}}^K) \mathbf{E}^{(0)}\end{aligned}$$

where  $\mathbf{E}^{(0)}$  is the initial embeddings at the embedding layer. The weights  $\alpha_l$  can either be trainable parameters or hand-picked values, the default setting is  $\alpha_l = \frac{1}{K+1}$ , i.e., uniform average. The affinity score for any (u, i) is the dot product of their final embeddings.

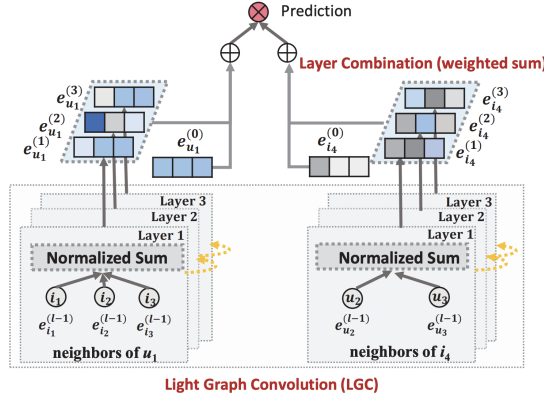


Figure 2.3: LightGCN architecture overview sourced from the original paper [? ].

### 2.2.3 Graph SAmple and aggreGatE (GraphSAGE)

Graph SAmple and aggreGatE (GraphSAGE) is a general framework primarily designed for inductive representation learning on graphs, i.e., training a generalised model capable of making predictions or embeddings on previously unseen nodes or entirely new graph structures. However, we observe that the underlying mechanisms of GraphSAGE can be effectively adapted for transductive learning scenarios, where the model learns representations within a fixed graph structure without encountering unseen nodes during inference [? ].

In our study, we leverage the sample and aggregate mechanism of GraphSAGE to learn embeddings within a fixed user-item interaction bipartite graph. This approach allows us to explore the framework’s potential for scaling up training while maintaining the representation learning capabilities of NGCF and LightGCN.

### Modified GraphSAGE Algorithm

We adapt the original GraphSAGE algorithm to suit our specific use case in NGCF and LightGCN. Our modified approach adheres the core sample and aggregate paradigm as in Figure 2.4: Firstly, sample  $k$ -hop neighbours from the target user  $u$  and item  $i$  nodes such that the resultant sub-graph has edges branching out from the target nodes, capturing the local structure. Secondly, we confine our aggregator functions to the sampled sub-graph instead of the full bipartite graph, improving training efficiency and model scalability.

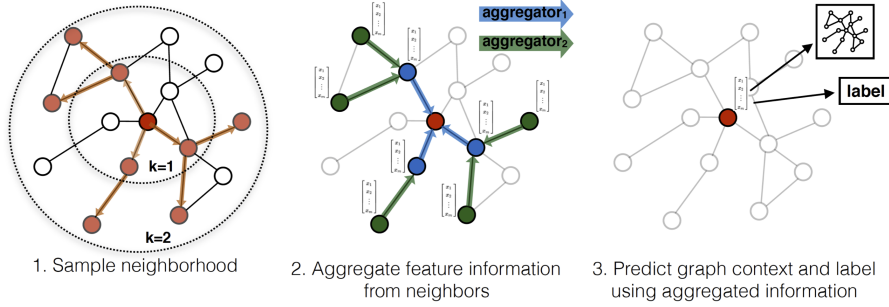


Figure 2.4: Visualisation of the GraphSAGE sample and aggregate mechanism from the original paper [? ].

The modified forward propagation algorithms NGCF-SAGE and LightGCN-SAGE are described in Algorithm 2. The algorithm closely resembles the original GraphSAGE structure while main differences are: Trainable node embeddings are used as input instead of fixed input features, embeddings propagation procedures specific to NGCF or LightGCN are employed instead of GraphSAGE aggregate functions, and embeddings in all layers will be used during the prediction stage instead of the embeddings in the final layer.

---

#### Algorithm 2: Modified algorithm for NGCF-SAGE and LightGCN-SAGE.

---

**Input:** Set of target nodes to compute embeddings  $\mathcal{V}$ , can be user and item nodes; initial node embeddings  $\{\mathbf{e}_v^{(0)}, \forall v \in \mathcal{V}\}$ ; depth  $K$ ; aggregator functions  $\text{AGGREGATE}_k, \forall k \in \{1, \dots, K\}$ ; prediction function  $\text{PREDICT}$ ; neighbourhood function  $\mathcal{N} : v \rightarrow 2^{\mathcal{V}}$  where  $v \in \mathcal{V}$

**Output:** Final embeddings  $\mathbf{e}_v^*, \forall v \in \mathcal{V}$

```

1  $\mathcal{E}_v \leftarrow \{\mathbf{e}_v^{(0)}\}, \forall v \in \mathcal{V}$ 
2 for  $k = 1$  to  $K$  do
3   for  $v \in \mathcal{V}$  do
4      $\mathbf{e}_v^{(k)} \leftarrow \text{AGGREGATE}_k(\mathbf{e}_v^{(k-1)}, \{\mathbf{e}_u^{(k-1)}, \forall u \in \mathcal{N}(v)\})$ 
5   end
6    $\mathbf{e}_v^{(k)} \leftarrow \mathbf{e}_v^{(k)} / \|\mathbf{e}_v^{(k)}\|_2, \forall v \in \mathcal{V}$  // Optional for LightGCN
7    $\mathcal{E}_v \leftarrow \mathcal{E}_v \cup \{\mathbf{e}_v^{(k)}\}, \forall v \in \mathcal{V}$ 
8 end
9  $\mathbf{e}_v^* \leftarrow \text{PREDICT}(\mathcal{E}_v), \forall v \in \mathcal{V}$ 

```

---

### NGCF-SAGE Details

For the aggregate function, we can consolidate the message construction and aggregation steps, as described in Section 2.2.1, into a single operation. This operation aggregates both self and neighbour embeddings, then applies model weights  $\mathbf{W}_1, \mathbf{W}_2 \in \Theta$  and an activation function. For each target node  $v \in \mathcal{V}$  at the  $k$ -th layer, we formally define  $\text{AGGREGATE}_k(\cdot)$ :

$$\begin{aligned} \mathbf{e}_v^{(k)} &= \text{AGGREGATE}_k(\mathbf{e}_v^{(k-1)}, \{\mathbf{e}_u^{(k-1)}, \forall u \in \mathcal{N}(v)\} | \Theta^{(k)}) \\ &:= \text{LeakyReLU}\left(\mathbf{W}_1^{(k)} \mathbf{e}_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} p_{vu} \left(\mathbf{W}_1^{(k)} \mathbf{e}_u^{(k-1)} + \mathbf{W}_2^{(k)} (\mathbf{e}_u^{(k-1)} \odot \mathbf{e}_v^{(k-1)})\right)\right) \end{aligned}$$

For mini-batch training, we employ matrix form aggregation for simultaneous embedding computation of a batch of nodes, facilitating GPU utilisation. Let  $\mathcal{V}$  be the set of target nodes in a mini-batch, and  $\mathcal{U}$  be the set of their sampled neighbourhood (non target nodes in the sub-graph). We can then construct a sub-graph adjacency matrix  $\tilde{\mathbf{A}} \in \mathbb{R}^{(|\mathcal{V}|+|\mathcal{U}|) \times (|\mathcal{V}|+|\mathcal{U}|)}$  and compute the corresponding normalised sub-graph Laplacian  $\tilde{\mathbf{L}}_{\text{norm}}$ . The matrix form NGCF aggregator function at layer  $k$  can then be expressed as:

$$\begin{aligned} \tilde{\mathbf{E}}^{(k)} &= \text{AGGREGATE}_k(\mathbf{E}_{\mathcal{V}}^{(k-1)}, \mathbf{E}_{\mathcal{U}}^{(k-1)} | \Theta^{(k)}) \\ &:= \text{LeakyReLU}\left(\tilde{\mathbf{L}}_{\text{norm}} \tilde{\mathbf{E}}^{(k-1)} \mathbf{W}_1^{(k)} + \tilde{\mathbf{L}}_{\text{norm}} \tilde{\mathbf{E}}^{(k-1)} \odot \tilde{\mathbf{E}}^{(k-1)} \mathbf{W}_2^{(k)}\right) \end{aligned}$$

where  $\tilde{\mathbf{E}} := \mathbf{E}_{\mathcal{V} \cup \mathcal{U}} \in \mathbb{R}^{(|\mathcal{V}|+|\mathcal{U}|) \times d}$  is the matrix of embeddings for all nodes in the sub-graph. It is crucial to update all embeddings in the sub-graph during the aggregation process to preserve the multi-hop message property of NGCF across multiple layers. Consequently, we modify line 1 and 7 of Algorithm 2 to collect target node embeddings at different layers:

$$\begin{aligned} \text{Line 1: } \mathcal{E}_{\mathcal{V}} &\leftarrow \{\mathbf{E}_{\mathcal{V}}^{(0)}\} \\ \text{Line 7: } \mathcal{E}_{\mathcal{V}} &\leftarrow \mathcal{E}_{\mathcal{V}} \cup \{[\tilde{\mathbf{E}}^{(k)}]_{\mathcal{V}}\} \end{aligned}$$

where  $[\cdot]_{\mathcal{V}}$  denotes the operation of selecting rows corresponding to target nodes  $\mathcal{V}$  that  $[\tilde{\mathbf{E}}]_{\mathcal{V}} = \mathbf{E}_{\mathcal{V}}$ . Now  $\mathcal{E}_{\mathcal{V}}$  stores the set of embeddings of each layer, we can simply define  $\text{PREDICT}(\cdot)$  of Algorithm 2 to return the full concatenated embeddings, same as the prediction layer as in Section 2.2.1.

### LightGCN-SAGE Details

For the aggregation function, we can easily adopt the same equation in the embeddings propagation layer as in Section 2.2.2.

$$\begin{aligned} \mathbf{e}_v^{(k)} &= \text{AGGREGATE}_k(\{\mathbf{e}_u^{(k-1)}, \forall u \in \mathcal{N}(v)\}) \\ &:= \sum_{u \in \mathcal{N}(v)} p_{vu} \mathbf{e}_u^{(k-1)} \end{aligned}$$

or in matrix form:

$$\begin{aligned}\tilde{\mathbf{E}}^{(k)} &= \text{AGGREGATE}_k(\mathbf{E}_{\mathcal{V}}^{(k-1)}, \mathbf{E}_{\mathcal{U}}^{(k-1)}) \\ &:= \tilde{\mathcal{L}}_{\text{norm}} \tilde{\mathbf{E}}^{(k-1)}\end{aligned}$$

where  $\tilde{\mathbf{E}} \in \mathbb{R}^{(|\mathcal{V}|+|\mathcal{U}|) \times d}$  is the embeddings matrix of the sub-graph,  $\tilde{\mathcal{L}}_{\text{norm}} \in \mathbb{R}^{(|\mathcal{V}|+|\mathcal{U}|) \times d}$  is the normalised sub-graph Laplacian without self-connection. Note that here we are also updating all embeddings in the sub-graph during aggregation, and only collect the set of embeddings of target nodes  $\mathcal{E}_{\mathcal{V}}$  for the prediction function, as mentioned previously. Then, we can define PREDICT( $\cdot$ ) of Algorithm 2 to return the weighted or uniform average of embeddings, same as the prediction layer as in Section 2.2.2.

### Sampling Mechanism

Our sampling technique closely follows the implementation described in the GraphSAGE paper. The process begins with the preparation of a deterministic neighbour function  $\mathcal{N}_k(v)$ , which pre-samples the neighbours of every node  $v$  at the  $k$ -th layer at the beginning of each epoch. This pre-computation allows for efficient lookup of neighbour samples when constructing sub-graphs for aggregation, eliminating the need for repeated sampling computations [? ].

The sampling scheme for minibatch training of a  $K$ -layer model begins by sampling a batch of users, each paired with positive and negative items. These sampled users and their associated items form the set of target nodes  $\mathcal{V}$ . For each target node  $v$  in  $\mathcal{V}$ , we apply the neighbour sampling function  $\mathcal{N}_u(v)$  to obtain its immediate neighbours  $u$ . We then recursively apply  $\mathcal{N}_{u'}(u)$  to these neighbours, sampling higher-hop neighbours  $u'$  until we reach the  $K$ -th hop, thereby obtaining the full set of neighbour nodes  $\mathcal{U}$  and completing the sub-graph. Throughout this process, we construct the adjacency matrix  $\tilde{\mathbf{A}}$  of the sub-graph. Finally, we compute the normalised Laplacian from  $\tilde{\mathbf{A}}$ , which is subsequently used in the aggregator functions.

## 2.3 Experimental Setup

### 2.3.1 Optimisation

#### Loss Function

The model parameters are optimised using the pairwise Bayesian Personalised Ranking (BPR) loss function, which is widely used in recommender systems [? ]. This approach considers the relative order between positive and negative user-item interactions, aligning with the core principle of recommendation tasks. The BPR loss is based on the assumption that observed interactions are more reflective of a user’s preferences. They should be assigned with higher prediction values than unobserved ones, this property is suitable for modelling implicit feedback. We follow the same definition of BPR as in NGCF and LightGCN [? ? ]:

$$\mathcal{L}_{\text{BPR}} = \sum_{(u,i,j) \in \mathcal{O}} -\ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|\Theta\|_2^2$$

where  $O$  represents the sampled set of pairwise training data,  $\sigma(\cdot)$  is the sigmoid function, and  $\Theta$  encompasses all trainable parameters. The term  $\lambda \|\Theta\|_2^2$  introduces L2 regularization, with  $\lambda$  controlling its strength.

There are also other loss functions for modelling implicit feedbacks. For example, the margin ranking loss  $L_{\text{Margin}} = \sum_{(u,i,j) \in O} \max(0, m - (\hat{y}_{ui} - \hat{y}_{uj}))$  is inspired from Support Vector Machine (SVM) and adapted for recommendation tasks. It aims to minimise the distance between relevant items and maximise the distance between irrelevant items. This loss only penalises when the difference between positive and negative predictions is smaller than the margin parameter  $m$ . More recent approaches such as the debiased Contrastive Loss further improves sampling quality and removes the sampling bias of BPR [? ].

### Optimisation Settings

We employ the Adam optimiser with minibatch training for all models. From hyperparameter search experiments, we found that a small, constant learning rate was sufficient for smooth convergence, eliminating the need for learning rate decay.

To generate training samples, we employ the naive negative sampling strategy. For each user, we sample one positive item from the relevant item pool (items which are interacted by the user in the train set) and one negative item from the irrelevant item pool (items outside the relevant pool). While more sophisticated techniques like hard negative sampling [? ] or debiased sampling [? ] could potentially enhance model training in production environments, we maintain simplicity for the purposes of this study.

We adopt the same definition of an epoch as in NGCF which the number of training samples is equivalent to the number of training edges in the graph, with user batches sampled uniformly with replacement [? ]. To improve training efficiency, we explore an alternative epoch definition where all users are iterated exactly once per epoch. The details will be discussed in Section 2.3.3.

To prevent overfitting and optimise computational resources, we implemented early stopping for hyperparameter search experiments and model evaluation experiments. Models are trained while tracking the validation set’s Rescaled Recall@20, with a tolerance of 25 epochs. The model checkpoint with the best validation performance will be selected as the final model.

### 2.3.2 Model Hyperparameters

Here we outline the key hyperparameters for each model and their usage, most of these hyperparameters will be fine-tuned during the hyperparameter search experiments.

#### Neural Graph Collaborative Filtering (NGCF)

The key hyperparameters for NGCF are as follows: **Embedding size** determines the dimensionality of user and item embeddings, with larger sizes increasing model complexity. **Number of layers** and **layer sizes** control the depth and width of the hidden layers during message propagation, with deeper and wider layers adding to the model complexity. **Regularization strength** refers to the magnitude of the penalty term in the loss function and is applied to all trainable parameters. **Learning rate** controls the step size during weight optimisation.

NGCF also employs two dropout techniques: **Node dropout** randomly removes nodes from the graph by setting entries of the normalised Laplacian to zero. **Message dropout** randomly excludes messages during the aggregation stage. Both techniques enhance model robustness and promote more generalised representations [? ].

### Light Graph Convolution Network (LightGCN)

LightGCN shares similar hyperparameters with NGCF: **Embedding size** defines the dimensionality of user and item representations. **Number of layers** controls the depth of message propagation, determining the furthest neighbours a node can reach. Unlike NGCF, LightGCN omits feature transformation and nonlinear activation, so “layer sizes” do not need specification. **Regularization strength** applies to the initial embeddings. **Learning rate** governs the optimisation step size. The weights in the prediction layer of LightGCN can be treated as hyperparameters but they are kept uniform in our study, as mentioned in Section 2.2.2.

While node and message dropout can be implemented similarly to NGCF, the simplicity of LightGCN with regularization alone is generally sufficient to prevent overfitting, making the model easier to train and fine-tune [? ].

### GraphSAGE

The integrated NGCF-SAGE and LightGCN-SAGE algorithms share most of its hyperparameters with NGCF and LightGCN but introduces an additional parameter: **Number of neighbour samples**  $S_k$  at layer  $k$ . This parameter controls the number of neighbours randomly sampled for each node during aggregation at each layer. It is crucial for GraphSAGE’s scalability, allowing efficient operation on large graphs by limiting the neighbourhood size. There is a trade-off between computational efficiency and the amount of structural information captured; larger sample sizes can capture more information but at the cost of increased computation and memory usage. Empirical studies suggest that two-hop neighbour sampling is optimal when  $S_1 \times S_2 \leq 500$  [? ].

## 2.3.3 Large-scale Model Training and Inference

In real-world recommender systems, handling large-scale datasets is a fundamental challenge. Consequently, efficient training and inference strategies are paramount to ensure both scalability and performance. For model training, mini-batch training via user and item sampling is typically preferred over iterating through the entire dataset. For model inference, various techniques are employed to optimise GPU memory usage while preserving computational efficiency.

### Training Strategies

We propose two different definitions of an epoch to optimise the training process. The choice between these two approaches depends on the trade-off between training time and training variance.

For original approach by NGCF, we define an epoch based on the number of training edges. Here, the number of training samples in an epoch is equivalent to the total number of training edges in the graph, expressed as number of batches per epoch  $\times$  batch size = number of training edges. User batches are sampled uniformly from the entire user pool with replacement between batches,

ensuring each training edge has an equal probability of being sampled in each epoch. An alternative strategy is to sample user nodes according to their degree, where the probability of being sampled is proportional to the number of edges as more complex purchase behaviors may require more training.

Another approach is based on the number of users, where number of batches per epoch  $\times$  batch size = number of users. Also, this approach guarantees that each user is considered exactly once per epoch, this approach allows much less number of batches per epoch, since usually number of user nodes  $\ll$  number of training edges, but it has a drawdown of higher training variance. To mitigate this, users can be swept through multiple times within an epoch, reducing training variance at the cost of increased epoch length and consequently longer training time per epoch.

Over the course of training, these two approaches are expected to converge towards similar performance. We conduct a comparative analysis to evaluate the efficiency and convergence characteristics of the two methods, with results presented in subsequent sections.

## Inference Strategies

We propose two main strategies which are compatible with NGCF and LightGCN: Full inference with  $n$ -fold division and partial inference with GraphSAGE.

For full inference, the primary computational bottleneck occurs during the matrix multiplication of the normalised Laplacian and the embedding matrix, which scales exponentially with graph size. To mitigate this, we partition the normalised Laplacian into  $n$  smaller sub-matrices. Each sub-matrix is then multiplied with the embedding matrix, and the results are aggregated to produce the full refined embeddings. This strategy enables efficient use of memory and computational resources, facilitating full-scale inference on large graphs. The choice of  $n$  depends on the available computational resources and the dataset size. In later sections, we will add suffix “- $nf$ ” to models trained with  $n$ -fold division, any models without suffix are treated as trained with 1-fold division, i.e. the normalised Laplacian is not divided.

For partial inference, we leverage the sampling mechanism inherent in the GraphSAGE model to avoid computing the full matrix multiplication of normalised Laplacian and embedding matrix. This strategy involves sampling a fixed number of neighbours for each target node, performing forward propagation only on the sampled sub-graph, and generating refined embeddings based on these partial computations. This approach significantly reduces the computational overhead during inference since we always perform message propagation on a fixed number of neighbours, making it particularly suitable for real-time recommendation with large-scale dataset, though we observed slight a trade-off in ranking performance and recommendation quality.

## 2.4 Evaluation

### 2.4.1 Ranking Metrics

To assess model performance rigorously, we employ a range of standard ranking metrics, including Recall@ $k$ , Rescaled Recall@ $k$ , Precision@ $k$ , Average Precision@ $k$  (AP@ $k$ ), and Normalised



Discounted Cumulative Gain@k (NDCG@k). These metrics provide a comprehensive evaluation framework, each offering distinct insights into various aspects of model effectiveness.

Recall is emphasized as primary metrics due to the large product base in the Tesco dataset, where it is crucial for the model to retrieve relevant products within the top recommendations. The focus on recall is particularly significant given the real-world application, where users are inclined to engage mainly with the top-ranked results.

Rescaled recall is also prioritised, particularly when comparing model performance across datasets of varying scales. This metric adjusts recall to account for differences in dataset size, ensuring meaningful comparisons across different evaluation scenarios.

To evaluate the impact of varying the recommendation cut-off, these metrics are assessed across different values of  $k$ , specifically  $k = 10, 20, 50, 100, 500$ . This range allows us to observe how recommendation quality changes as the window of top recommendations expands. While multiple  $k$  values are analysed, the primary focus is on  $k = 20$ , aligning with user behavior which typically concentrates on the top few results. This choice is also studied in the original NGCF and LightGCN papers, providing a consistent and comparable benchmark.

## 2.4.2 Downstream Tasks

### Personalised Offers

For the personalised offers downstream task, the affinity scores generated by the base ranking model are used to recommend the top personalised offers. Figure 2.5 provides an overview of the downstream model, where the ranking model’s affinity scores contributes one set of input features and the other set of input features is derived from other proprietary data. The downstream model will be retrained based on these input features and learn to output a ranked list of personalised offers.

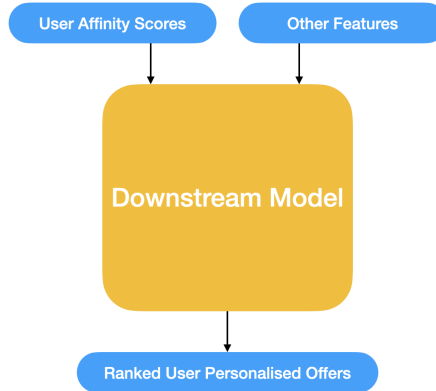


Figure 2.5: High-level diagram of the Personalised Offers downstream model.

Our study compares the downstream task performance of non graph-based models and GNN-based models. For non graph-based CF models, we include the current Tesco proprietary Alternating Least Squares (ALS) model. For GNNs, we train NGCF and LightGCN models with full inference on both small-scale and large-scale datasets.

The quality of personalised offers of different models are evaluated using Precision@4. This metric focuses on the top four recommendations for each user, providing insights into the models' ability to rank relevant offers highly. We aim to quantify the improvements offered by graph-based neural network approaches over traditional methods in the context of downstream task.

### 2.4.3 Embedding Space Analysis

#### Clustering Analysis

To assess the quality of the graph embeddings, we use distance-based methods to evaluate how effectively the model captures similarities between item or user groups. We apply either k-means clustering on the graph embeddings or known product hierarchies from the product information dataset to map items into predefined groups. Cluster coherence is then evaluated using the mean Silhouette score, revealing the model's ability to form meaningful clusters.

The Silhouette score for the  $i$ -th embedding is calculated as follows [? ]:

$$\text{Silhouette}(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where  $a(i)$  represents the average Euclidean distance from point  $i$  to all other points within the same cluster, and  $b(i)$  is the minimum average Euclidean distance from point  $i$  to all points in the nearest neighbouring cluster. The Silhouette score ranges from -1 to +1 where a score of +1 indicates that the point is well-clustered and closer to points within its own cluster than to points in other clusters, 0 indicates that the point lies near the decision boundary between two neighbouring clusters, -1 suggests that the point may be misclassified and is closer to points in a neighbouring cluster than to those in its own cluster.

#### Sparsity Analysis

Embedding sparsity indicates the model's efficiency and interpretability. We measure it using the Hoyer sparsity measure, which ranges from 0 being fully dense to 1 being fully sparse. We should expect fully sparse embeddings are perfect encoders of information since each dimension of the embedding space is fully utilised for a distinct latent feature or class, i.e., for each unique class, its embeddings should only put all magnitudes in only one dimension. Dense embeddings mean the latent space is overlapped and information of the same latent feature or class is spread across different dimensions.

For an embedding vector  $\mathbf{y} \in \mathbb{R}^d$ , the Hoyer sparsity is defined as [? ]:

$$\text{Hoyer}(\mathbf{e}) = \frac{\sqrt{d} - \frac{\|\mathbf{e}\|_1}{\|\mathbf{e}\|_2}}{\sqrt{d} - 1}$$

The mean sparsity across users can be calculated as:

$$\text{Sparsity} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \text{Hoyer}(\bar{\mathbf{e}}_u)$$

where each dimension of  $\mathbf{e}_u$  is normalised by the standard deviation  $\sigma_d$  across all users, i.e.,

$\bar{e}_{u,d} = \frac{e_{u,d}}{\sigma_d}$ , this normalises the length scale of each dimension and avoid embeddings reaching apparent high sparsity by varying length scales in different dimension [? ]. This calculation can be similarly applied to item embeddings.

## 2.4.4 Large-scale Training Analysis

### Training Strategies

We explore two definitions of an epoch to optimise training on large datasets as mentioned in Section 2.3.3. The first approach defines an epoch by the number of training edges in the graph, ensuring balanced sampling across all user-item interactions. The second approach defines an epoch by the number of users, effectively sweeping through all users once per epoch.

We compare the convergence rates and computational efficiency of these strategies and demonstrate that they are equivalent under certain conditions. We present empirical results highlighting the trade-offs between training time, model performance, and convergence stability, offering guidelines for choosing the most suitable strategy based on specific use cases.

### Inference Strategies

We analyse two approaches for large-scale inference as mentioned in Section 2.3.3. The full inference approach using n-fold Laplacian matrix decomposition and the partial inference approach using the GraphSAGE sampling technique.

We compare these methods in terms of inference speed, accuracy and scalability, with empirical results showing their performance across different graph sizes. This comparison provides insights into their applicability for real-time recommendations and batch processing.

## Chapter 3

# Result Analysis

### 3.1 Reproducing Yelp2018 Ranking Metrics

We reproduced the performance of the NGCF and LightGCN models on Yelp2018, closely following the recommended hyperparameters from the original implementations. For both models, we used an embedding size of 64, three layers (with layer sizes of 64 for NGCF), a learning rate of  $1 \times 10^{-3}$ , a regularization strength of  $1 \times 10^{-4}$ , with an additional message dropout rate of 0.1 for NGCF.

Table 3.1 compares the original reported results with our reproduced results on the test set and verifies our implementations’ accuracy. For the minor discrepancies in performance, it could be the randomness in data processing, slight differences in the training process or implementation details.

Table 3.1: Original and reproduced test set performance of 3-layer NGCF and LightGCN.

| Model            | Implementation | Recall@20 | NDCG@20 |
|------------------|----------------|-----------|---------|
| 3-layer NGCF     | Original       | 0.0579    | 0.0477  |
|                  | Reproduced     | 0.0558    | 0.0442  |
| 3-layer LightGCN | Original       | 0.0639    | 0.0525  |
|                  | Reproduced     | 0.0654    | 0.0526  |

### 3.2 Hyperparameter Search

To efficiently fine-tune hyperparameters within limited computational resources, we conducted a two-stage search using the small-scale Tesco-5k dataset as in Section 2.1.3. Firstly, a random search was performed over a broad range of hyperparameters to identify promising regions in the parameter space, allowing us to quickly eliminate poor configurations. This was followed by a more focused grid search within the more refined parameter ranges, enabling precise tuning while keeping certain degree of thoroughness in a larger search space.

The optimisation objective was Rescaled Recall@20 on the validation set, chosen for its robustness across different dataset sizes. Although the optimal values obtained from Tesco-5k may not be fully representative for larger datasets, the yielded hyperparameters were found to be relatively robust in other experiments.

For NGCF, the initial search space included embedding sizes  $\{64, 128\}$ , number of layers  $\{2, 3, 4\}$  with fixed sizes of 64, regularization strengths  $\{1 \times 10^{-5}, \dots, 1 \times 10^{-1}\}$ , learning rates  $\{1 \times 10^{-4}, 5 \times 10^{-4}, 1 \times 10^{-3}, 5 \times 10^{-3}, 1 \times 10^{-2}\}$ . For LightGCN, the initial search space included embedding sizes  $\{64, 128, 256, 512\}$ , number of layers  $\{2, 3, 4, 5\}$ , regularization strengths  $\{1 \times 10^{-5}, \dots, 1 \times 10^{-1}\}$ , and learning rates  $\{1 \times 10^{-4}, 5 \times 10^{-4}, 1 \times 10^{-3}, 5 \times 10^{-3}, 1 \times 10^{-2}\}$ .

Table 3.2: Optimal hyperparameters found on Tesco-5k dataset.

| Hyperparameters               | NGCF               | LightGCN           |
|-------------------------------|--------------------|--------------------|
| Embedding size                | 64                 | 512                |
| Number of Layers              | 2                  | 3                  |
| Layer size                    | 64                 | -                  |
| Regularization Strength       | $1 \times 10^{-5}$ | $1 \times 10^{-5}$ |
| Learning Rate                 | $1 \times 10^{-3}$ | $5 \times 10^{-4}$ |
| Validation Rescaled Recall@20 | 0.13377            | 0.13918            |

Note that these optimal hyperparameters are also adapted for GraphSAGE variants. However, the number of neighbour samples  $S_k$  hyperparameter for SAGE is not optimised through hyperparameter search. We follow the empirical total neighbour samples  $\leq 500$  rule as suggested in the GraphSAGE paper [?], setting the number of neighbour samples of 2-layer NGCF-SAGE as  $\{S_1, S_2\} = \{25, 20\}$  and 3-layer LightGCN-SAGE as  $\{S_1, S_2, S_3\} = \{10, 10, 5\}$ .

### 3.3 Ranking Performance

#### 3.3.1 Tesco-20k

We trained and evaluated the models on the medium-sized Tesco-20k dataset to assess their generalisation ability. This evaluation includes full inference methods (NGCF and LightGCN) and partial inference methods (NGCF-SAGE and LightGCN-SAGE) using the hyperparameters listed in Table 3.2. To ensure convergence, we defined epochs based on the number of training edges and trained the models with early stopping based on validation Rescaled Recall@20.

Table 3.3 highlights the performance differences across models. LightGCN shows the highest improvement over the MF baseline, with about 9% increase in Recall metrics and about 6% increase in NDCG. NGCF also demonstrates performance gains, with about 5% increase in Recall metrics and about 2.5% increase in NDCG. These results suggest GNN-based models can indeed capture more collaborative signals than non graph-based model.

Partial inference methods showed some decline compared to full inference methods but still outperform the baseline to some extent. Note that the decline in performance for NGCF-SAGE is much more than LightGCN-SAGE. LightGCN-SAGE has a 20% drop in recall metrics and 10% drop in NDCG, this indicates the generalisation ability of LightGCN is strong as the collaborative signal learnt from the subgraph is also capturing most of the information in the full graph. On the other hand, the 60% drop in all performance of NGCF-SAGE suggests that NGCF-SAGE may require better neighbourhood sampling strategy and more samples to learn the general representation of the full graph, or NGCF may not be easily compatible with SAGE.

Table 3.3: Tesco-20k test set ranking metrics of different models trained until convergence.

| Model                | Recall@20 <sup>†</sup> | Rescaled Recall@20     | NDCG@20                |
|----------------------|------------------------|------------------------|------------------------|
| <b>MF (Baseline)</b> | 0.0553                 | 0.1153                 | 0.1461                 |
| <b>NGCF</b>          | 0.0580 (+4.88%)        | 0.1211 (+5.03%)        | 0.1500 (+2.67%)        |
| <b>NGCF-SAGE</b>     | 0.0564 (+1.99%)        | 0.1178 (+2.17%)        | 0.1474 (+0.89%)        |
| <b>LightGCN</b>      | <b>0.0604 (+9.22%)</b> | <b>0.1260 (+9.28%)</b> | <b>0.1545 (+5.75%)</b> |
| <b>LightGCN-SAGE</b> | 0.0594 (+7.41%)        | 0.1241 (+7.63%)        | 0.1535 (+5.07%)        |

<sup>†</sup>Recall@20 should not be used for comparing performance across datasets of different scales.

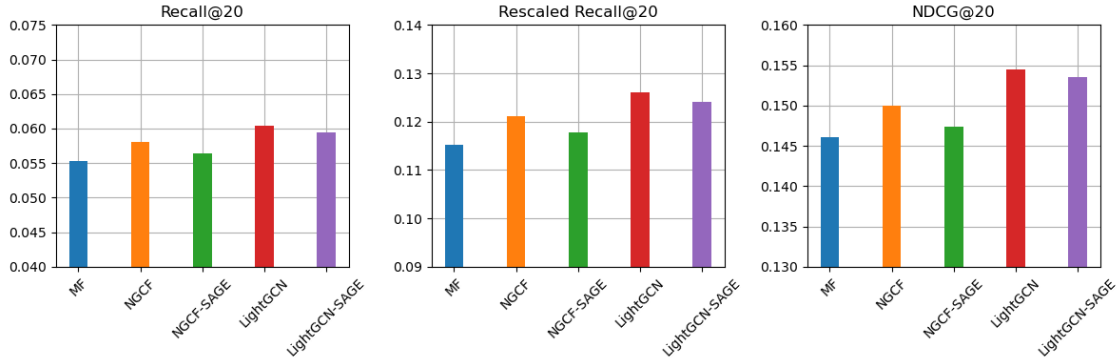


Figure 3.1: Tesco-20k test set ranking metrics of different models trained until convergence.

### 3.3.2 Tesco-228k

We further applied the models to the Tesco-228k dataset to evaluate their capabilities in handling big data scenarios. We used the same hyperparameters from Table 3.2 and defined epochs based on the number of training edges for better convergence. Due to the significant computational time and cost, all models were limited to 10 training epochs and 5-fold division was applied for full inference methods. The results in Table 3.4 show how each model performs at larger scale.

NGCF demonstrates the strongest performance, with an increase of 11%-12% across all metrics. LightGCN shows less improvement, achieving a 6%-7% increase in all metrics. The stronger improvement NGCF in Tesco-228k dataset indicates that the feature transformation and non-linearity become more effective with increased data volume, that the modelling capabilities of NGCF is higher than LightGCN.

For partial inference variants, both NGCF-SAGE and LightGCN-SAGE exhibit performance decline compared to their respective full inference methods, with NGCF-SAGE showing a much significant decline than LightGCN-SAGE. LightGCN-SAGE has about 20% drop in recall metrics and same performance in NDCG, while NGCF-SAGE has more than 90% drop in all metrics. This is similar to the phenomenon observed in Tesco-20k from Section 3.3.1, suggesting there are further rooms to improve NGCF-SAGE in terms of sampling strategy or architecture design.

In general, if we compare the Rescaled Recall of all models in Tesco-228k to the ones in Tesco-20k, the scores tripled for large-scale models. This reflects big data has an overall positive effect to collaborative filtering models since there are much more user-item interactions to learn from.

Table 3.4: Tesco-228k test set ranking metrics of different models trained for 10 epochs.

| Model         | Recall@20 <sup>†</sup>  | Rescaled Recall@20      | NDCG@20                 |
|---------------|-------------------------|-------------------------|-------------------------|
| MF (Baseline) | 0.0306                  | 0.3316                  | 0.3710                  |
| NGCF-5f       | <b>0.0343 (+12.09%)</b> | <b>0.3693 (+11.37%)</b> | <b>0.4130 (+11.32%)</b> |
| NGCF-SAGE     | 0.0310 (+1.31%)         | 0.3350 (+1.03%)         | 0.3721 (+0.30%)         |
| LightGCN-5f   | 0.0328 (+7.19%)         | 0.3530 (+6.45%)         | 0.3931 (+5.96%)         |
| LightGCN-SAGE | 0.0322 (+5.23%)         | 0.3485 (+5.10%)         | 0.3929 (+5.90%)         |

<sup>†</sup>Recall@20 should not be used for comparing performance across datasets of different scales.

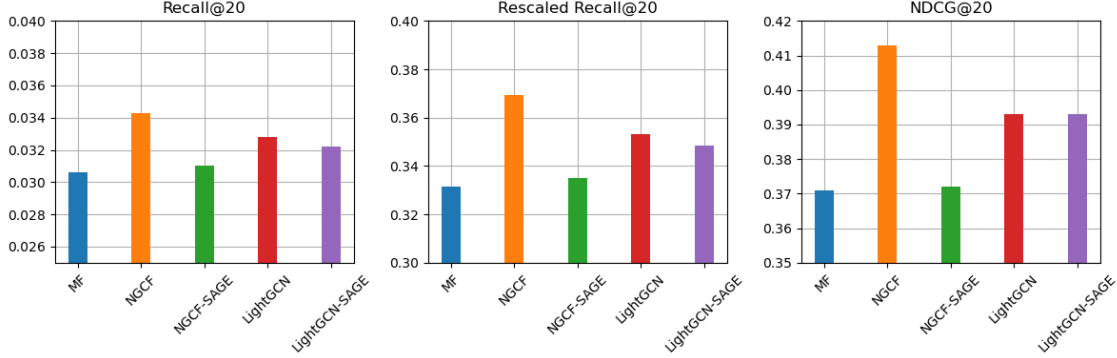


Figure 3.2: Tesco-228k test set ranking metrics of different models trained for 10 epochs.

### 3.4 Downstream Tasks Performance

For the personalised offers downstream task, we used different sets of affinity scores generated by NGCF and LightGCN as the partial input while keeping all other input features constant for retraining of the downstream model as illustrated in Figure 2.5. This ensures the downstream performance is only impacted by the quality of affinity scores. On the other hand, Tesco provided the downstream performance of using affinity scores from a proprietary Alternating Least Squares (ALS) model, serving as a strong non-graph matrix factorisation baseline.

The evaluation pipeline also included a universal baseline using a non-personalised ranking based on the Global Popularity (GP) of products. It is important to note that this precision scores differ across models, likely due to variations in the product sets used in the pipeline. Therefore, it is more fair to use the improvement over GP ratio when comparing across models.

Table 3.5 presents the performance of the personalised offers generated using the affinity scores of different models. LightGCN trained by Tesco-20k achieved the highest Precision@4 of 0.4661 followed by NGCF-5f trained by Tesco-228k of 0.4354. However, the raw precision scores are not truly reliable since the evaluation pipeline may not put these models on the common ground.

In terms of improvement ratio over their respective baselines, the GNN-based models trained on Tesco-20k showed about 20% less improvements than the ALS baseline, while the GNN-based models trained on Tesco-228k gained about 50% improvements than the ALS baseline. This suggests GNN-based models in large-scale limit are possible to reach superior performance than non-graph based models.

Table 3.5: Comparison of personalised offers performance across different models.

| Dataset     | Model       | Precision@4 | GP Precision@4 | Improv. Ratio <sup>†</sup> |
|-------------|-------------|-------------|----------------|----------------------------|
| Proprietary | ALS         | 0.3236      | 0.1176         | 175.17%                    |
| Tesco-20k   | NGCF        | 0.4258      | 0.1786         | 138.41% (-20.99%)          |
|             | LightGCN    | 0.4661      | 0.1863         | 150.19% (-14.26%)          |
| Tesco-228k  | NGCF-5f     | 0.4354      | 0.1223         | 255.96% (+45.12%)          |
|             | LightGCN-5f | 0.4044      | 0.1077         | <b>275.31% (+57.17%)</b>   |

<sup>†</sup>The proprietary ALS model is served as baseline when computing the percentage changes in improvement ratios.



Figure 3.3: Comparison of personalised offers performance across different models.

## 3.5 Embedding Space Analysis

In this section, we analyse the user and item embedding space learnt by LightGCN trained on Tesco-20k. The embedding quality are evaluated in terms of clustering property and sparsity.

### 3.5.1 Clustering Analysis

#### Item Embeddings

We leveraged Tesco commercial hierarchies in Secion 2.1.1 to group products according to their classifications within each hierarchy, then we checked if the item embeddings resembled these groupings naturally through clusters. The resemblance of grouping effect in embedding space was assessed using the mean Silhouette score. We also applied k-means clustering on item embeddings and evaluated the mean Silhouette score at different k values to measure the clustering property of item graph embeddings.

Figure 3.4 (a) presents the mean Silhouette scores for grouping item embeddings under the five different levels of hierarchies. The results show slightly negative Silhouette scores, indicating that the item embeddings are not grouped to match the predefined hierarchies in the 512-d space.

Figure 3.4 (b) shows that item embeddings do not naturally form any clear-cut clusters in the 512-d space at different  $k$  values, since consistently low Silhouette scores about 0 are still observed and the mean score generally goes down when number of cluster increases.

Despite the low mean Silhouette score of item embeddings clustered from different Tesco com-



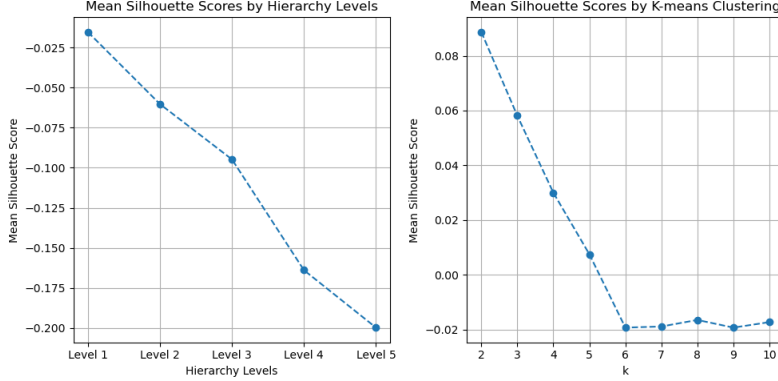


Figure 3.4: Mean Silhouette score by assigning item embeddings into categories according to Tesco commercial hierarchy (a, left), and clustering item embeddings using k-means (b, right). Item embeddings are generated by LightGCN trained on Tesco-20k.

mercial hierarchies, we proceeded to check the detailed Silhouette score distribution of each category in the level 1 hierarchy since it has the smallest number of categories, i.e., the level 1 categories are the broadest. It should be easier to form non-overlapping clusters as indicated by its relatively highest mean Silhouette score in Figure 3.4.

Figure 3.5 illustrates the Silhouette score of each item in the dataset, they are grouped by in level 1 hierarchy’s category and sorted by Silhouette score in descending order. Qualitatively, from these distributions, we observe that some categories like category 1, 2, 3, 4, 6 contain a larger portion of relatively “better” clustered data points, i.e., the portion with positive Silhouette scores. These categories seem to be relatively well-clustered in the embedding space compared to other categories with lower overall Silhouette scores like category 0, 5, 7. We can then examine the clustering quality difference between categories with better scores and lower scores.

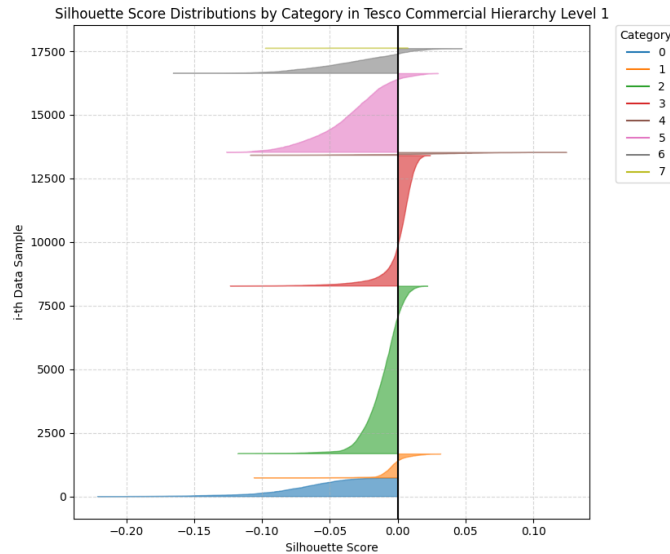


Figure 3.5: Silhouette score per sample sorted in descending order, by assigning item embeddings into categories according to Tesco commercial hierarchy level 1. Item embeddings are generated by LightGCN trained on Tesco-20k.

To visualise the clustering quality measured by the Silhouette score analysis in Figure 3.5, we further selected the better clustered category 3 and the worse clustered category 0 for further comparison.

Figure 3.6 shows the Uniform Manifold Approximation and Projection (UMAP) [?] of item embeddings from category 0 and 3. The UMAP embeddings lie on two major separated regions as observed, this indicates some kind of weak clustering effect and the model did learn to discriminate products to some extent. We then applied a two-component Gaussian Mixture Model to learn and determine the decision boundary between these two regions, as coloured by red and blue in the figure.

Table 3.6 shows the ratio of data points in each region for category 0 and 3. We find out the category 0 item embeddings are more concentrated in the red region while the category 3 item embeddings are more concentrated in the blue region. Interestingly, if we analyse all categories in hierarchy level 1, we find that food categories tend to centralise in the red region while non food categories tend to centralise in the blue region, the full results are available in Figure B.1 and Table B.1. In general we observed three patterns, the item embeddings of any category can be centralised in either region or evenly split across both regions based on the food versus non food nature of categories. These patterns group different user purchase behaviours on categories within the level 1 hierarchy.

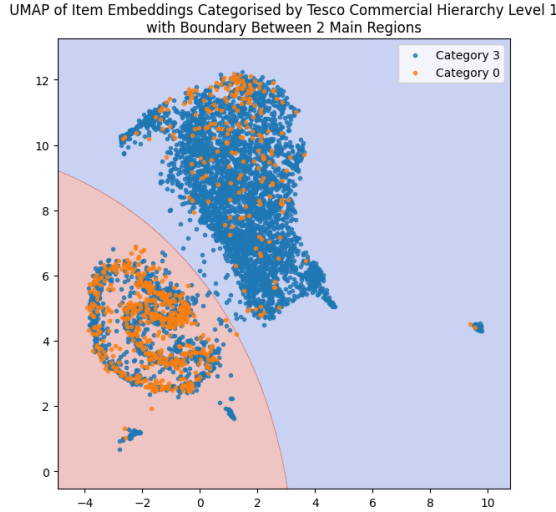


Figure 3.6: UMAP of item embeddings of category 0 and 3 from hierarchy level 1, generated by LightGCN trained on Tesco-20k. The decision boundary of the two regions is learnt by a 2-component Gaussian Mixture Model.

Table 3.6: Ratio of UMAP item embeddings falling into red or blue region in Figure 3.6.

| Hierarchy Level 1<br>Category | Ratio      |             |
|-------------------------------|------------|-------------|
|                               | Red Region | Blue Region |
| 0                             | 0.7145     | 0.2855      |
| 3                             | 0.2137     | 0.7863      |

## User Embeddings

We can assess whether natural clusters exist by calculating the mean Silhouette scores after applying k-means clustering with varying  $k$ . Figure 3.7 presents the mean Silhouette scores are generally slightly above 0, indicating very weak of clusters formation in 512-d embedding space.

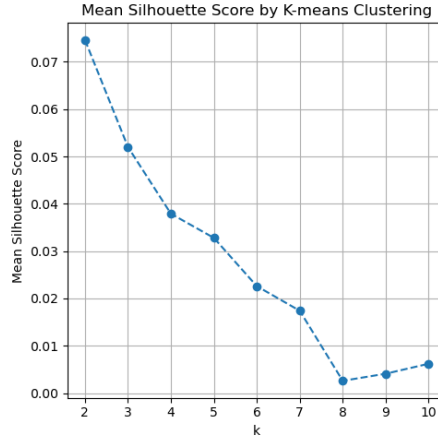


Figure 3.7: Mean Silhouette score by clustering user embeddings using k-means. Item embeddings are generated by LightGCN trained on Tesco-20k.

Further analysis on UMAP of user embeddings discovers one larger cluster and one small cluster as in Figure 3.8, a two-component Gaussian Mixture Model is applied to learn and determine the decision boundary between the clusters. Then we count the categories of items bought between these 2 clusters of users, there are no significant differences between the clusters across all 5 levels of hierarchy. Thus the user embeddings are not reflecting user purchasing behaviours according to Tesco commercial hierarchy.

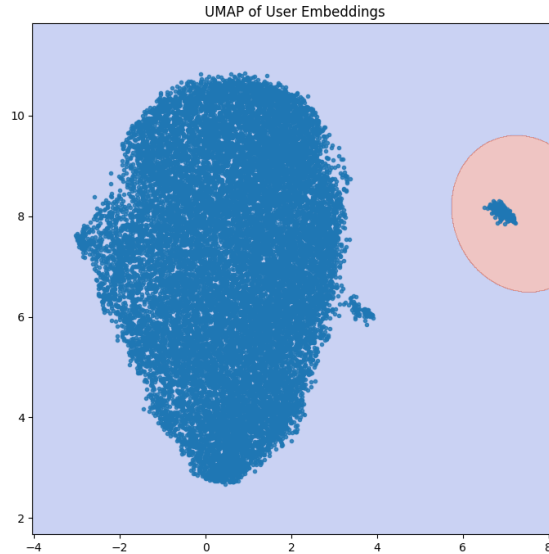


Figure 3.8: Mean Silhouette score by clustering user embeddings using k-means. Item embeddings are generated by LightGCN trained on Tesco-20k.

The clustering analysis of both user and item graph embeddings suggests that these embeddings were weakly clustered in the 512-d space. We showed that the item embeddings resemble the level 1 hierarchy to some extent, while the user embeddings exhibit limited clustering property which remains unexplained by Tesco commercial hierarchies. The relatively weak clustering properties of embeddings could be due to the data complexity or GNN-based models do not inherently learn to separate different groups of users and items due to architecture design or loss function formulation.

### 3.5.2 Sparsity Analysis

Sparsity analysis is another approach to assess embeddings quality, which represents the level of disentanglement of embeddings. Table 3.7 shows the mean Hoyer sparsity of all user embeddings and all item embeddings.

Table 3.7: Mean sparsity values for user and item embeddings generated by LightGCN trained on Tesco-20k.

| Embedding Type | Mean Sparsity |
|----------------|---------------|
| User           | 0.1883        |
| Item           | 0.1732        |

The relatively low sparsity observed in the graph embeddings suggests that the embeddings are mostly entangled, the model has limited ability to separate different latent factors into distinct dimensions.

#### Item Embeddings

We again assigned item embeddings into different categories according to Tesco commercial hierarchy level 1 same as the cluster analysis. We visualised the mean item embeddings of category 0 and 3 to examine the characteristics of different latent dimensions.

Figure 3.9 illustrates the magnitude of each dimension of the mean item embeddings. We observe that embeddings are generally spread across dimensions for both categories. Category 0 has varying magnitudes and occasional small spikes, while category 3 has a more consistent and even magnitude of 0.05. This observation agrees with the sparsity values presented in Table B.2, where category 0 embeddings have higher sparsity than category 1 embeddings. The full plot for all categories in hierarchy level can be found in Figure B.2.

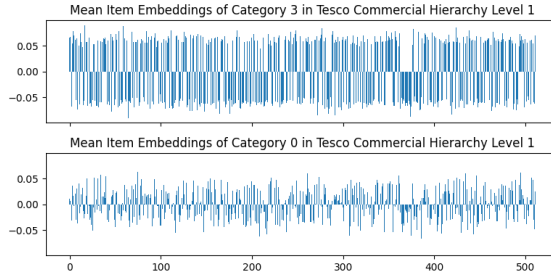


Figure 3.9: Magnitude of each dimension in the mean item embeddings (512-d) of category 0 and 3 from hierarchy level 1, generated by LightGCN trained on Tesco-20k.

## User Embeddings

Figure 3.10 illustrates the magnitude of each dimension of the mean embeddings, it has a more consistent and even magnitude of 0.1.

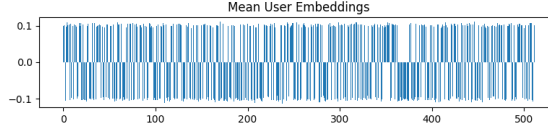


Figure 3.10: Magnitude of each dimension in the mean user embeddings (512-d), generated by LightGCN trained on Tesco-20k.

The sparsity analysis indicates that the graph embeddings are mostly dense with information mostly spread out in different latent dimension, this may have a linkage to the clustering analysis causing the embeddings not being well-clustered. The learnt representations from GNN-based models are dense and overlapped that are difficult to be distinctly separated in Euclidean space. However, further analysis on models trained with larger datasets and more refined hyperparameter tuning should be done before drawing a solid conclusion.

## 3.6 Scalability Analysis

### 3.6.1 Training Strategies Analysis

In this section, we evaluate the impact of different epoch definitions for training LightGCN with Tesco-20k. The two epoch definitions discussed in training strategies of Section 2.3.3 were tested: one based on sweeping through users and another based on uniform sampling of users according to the number of training edges.

We investigated the effect of varying user sweeps per epoch (1, 4, 8, 16) compared to uniform user sampling. Figure 3.11 shows the learning curves with early stopping based on validation Rescaled Recall@20 with a fixed patience of 25 epochs.

For the training loss curves in Figure 3.11 (a), we can treat uniform sampling as the “complete curve” as it performed the most training. The 16-sweep curve passed the flat basin of the “complete curve” and the loss kept decreasing, while 8 and 4-sweep curves stopped at the flat basin. The 1-sweep curve stopped between the initial big drop and the flat basin.

For the validation Rescaled Recall@20 curves in Figure 3.11 (b), we can perform similar analysis. The 16-sweep curve reached the global maximum of the “complete curve” and stopped at the onset of overfitting. The 8-sweep curve reached the first local maximum but couldn’t reach the global maximum. The 4 and 1-sweep curves stopped before finding the first local maximum.

These curves demonstrate how increasing user sweeps effectively approaches the “complete” uniform sampling curve, which is expected assuming all other hyperparameters are the same. Different sweep counts will reach the same optimum as in uniform sampling given that the early stopping patient is scaled respectively.

These findings indicate that the epoch definition primarily influences the frequency of model evaluation rather than directly impacting training speed. User sweeping appears faster due to fewer training samples per epoch, but it introduces higher training variance from fewer optimisation

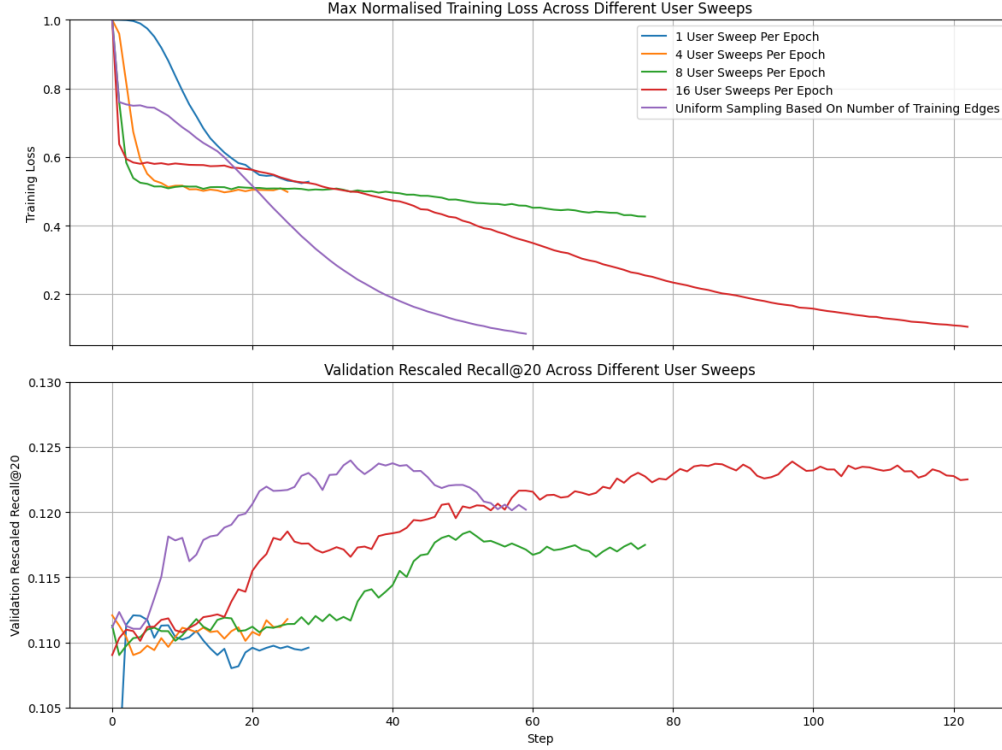


Figure 3.11: Max normalised training loss curves comparing different epoch definitions (a, top), and Rescaled Recall@20 curve on validation set comparing different epoch definitions (b, bottom)

updates and more frequent evaluations. Conversely, increasing sweeps per epoch or using uniform sampling reduces variance at the cost of longer epochs. Other than user sweeping, we can have an alternative definition based on subset of training edges rather than all edges, which can achieve similar effects of shorter epochs with increased training variance.

While our proposed epoch definitions do not necessarily accelerate training, they do affect the granularity of performance monitoring analogous to adjusting the resolution of the training curve. This could be useful for more precise early stopping and increase training efficiency. For example, the Rescaled Recall curve in Figure 3.11 (b) suggests 16 user sweeps can potentially stop earlier than uniform sampling and result in a similarly performing model. The results indicate that the training convergence speed of user sweeping and uniform sampling are comparable. However, there exist better sampling strategies and epoch definitions that could enhance training convergence.

### 3.6.2 Training Speed Analysis

We analyzed the two large-scale inference strategies from Section 2.3.3, namely n-fold division and GraphSAGE. Table 3.8 and Figure 3.12 shows different batch size settings and training speeds per batch for both strategies, the computation time is measured on a virtual machine with 4 vCPU cores, 28GB Memory and a NVIDIA Tesla T4 GPU paired with 16GB GPU Memory.

For the smaller Tesco-20k dataset, two methods perform similarly, with GraphSAGE slightly faster than 1-fold division, i.e., no matrix division, note that Tesco-20k has a small memory footprint that allows full matrix product.

For the larger Tesco-228k dataset, GraphSAGE’s computation time scales much less than 5-fold division, note that 5-fold is already optimal for balancing computation speed and memory constraints. This demonstrates the effectiveness of GraphSAGE’s fixed-neighbour sampling approach compared to computing  $n$  submatrix products. The computational speed of  $n$ -fold matrix division decreases significantly as  $n$  increases, which is inevitable as the graph gets larger.

Table 3.8: Average computation time (seconds) per batch across different batch sizes and inference strategies. Entries with asterisk (\*) indicate the actual settings used in our experiments.

| Dataset    | Model         | Batch size   |              |       |               |
|------------|---------------|--------------|--------------|-------|---------------|
|            |               | 1024         | 4096         | 10240 | 20480         |
| Tesco-20k  | LightGCN      | <b>1.46*</b> | 2.25         | 4.06  | 10.33         |
|            | LightGCN-SAGE | <b>1.34*</b> | 1.55         | 3.22  | 9.31          |
| Tesco-228k | LightGCN-5f   | 21.69        | 25.78        | 33.59 | <b>47.27*</b> |
|            | LightGCN-SAGE | 2.97         | <b>7.24*</b> | 16.34 | 28.52         |

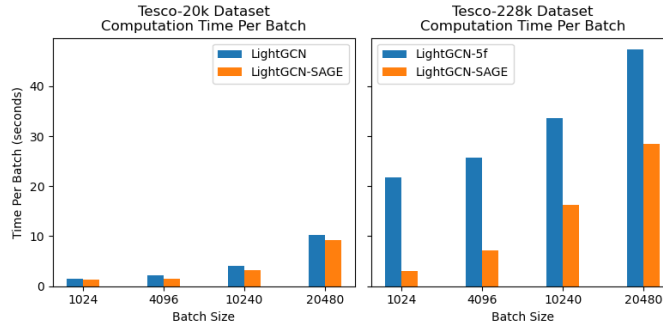


Figure 3.12: Average computation time (seconds) per batch across different batch sizes and inference strategies.

In our experiments, we generally prefer smaller batch sizes when feasible. Smaller batch sizes introduce more noises in gradient estimates and help explore the loss landscape more effectively, potentially escaping local minima and improve generalisation. They also allow frequent parameter updates and help convergence in terms of the number of epochs. For GraphSAGE specifically, minibatches are essential to avoid sampling excessively large sub-graphs.

For Tesco-20k, we select a small batch size of 1024 for both methods as they have comparable training times. For Tesco-228k, a larger batch size of 20480 is chosen for the 5-fold matrix division to minimise training time per epoch. In contrast, a relatively small batch size of 4096 is chosen for GraphSAGE, balancing training speed with the method’s inherent need for minibatches. Consequently, for both datasets, the training time per epoch of both methods is similar, with GraphSAGE being slightly faster than  $n$ -fold matrix division.

We believe that for even larger datasets, GraphSAGE will be the only feasible solution to balance memory consumption and training speed. It is worth to note that while GraphSAGE is faster and scales better with large dataset, it shows decline in ranking performance as in Table 3.3 and Table 3.4. To alleviate the decline, we hypothesise a hybrid approach of partial inference in training and full inference in evaluation will be optimal in balancing computational constraints and embedding quality.

## Chapter 4

# Method Extension

### 4.1 More Large-Scale Training Techniques

In our study, we observed that the speed bottleneck in large-scale training of can be resolved by parallelisation. In general, data samples within a training batch can be split and distributed across multiple GPUs, enabling faster computation and supporting even larger batch sizes.

For the  $n$ -fold division strategy, the computation of  $n$  sub-matrix products during the forward pass can also be parallelised, providing an additional option for GPU parallelisation. For the SAGE strategy, the neighbourhood sampling stage can be parallelised across CPUs, which can be combined with the aforementioned GPU parallelisation of the training batch.

Additionally, other research such as PinSage has introduced several techniques for optimising large-scale training and deployment of GNN-based recommendation models. Firstly, instead of using fixed-size uniform neighbour sampling like GraphSAGE, PinSage employs random walks to sample a higher quality sub-graph. This approach computes importance scores for each node on-the-fly, which will be used in their pooling and aggregation step. Secondly, PinSage samples a batch of users and shares 500 common negative samples within the batch, such that each user effectively pairs with more negative samples. Also, curriculum training is applied where more hard negative examples are sampled as training progresses, leading to faster convergence and improved model generalisation. Lastly, PinSage utilises a MapReduce pipeline to distribute the trained model for generating embeddings for billions of nodes during inference, minimising repeated computations [?].

### 4.2 Incorporating Node Features

#### Product Categorical Features

During our exploration of the original GraphSAGE framework, we attempted to train GNNs with only node features but no initial node embeddings. The model learns parameters within the aggregation layers and generates new graph embeddings for each node after aggregation.

As a result, we constructed a homogeneous graph with only node features based on Tesco-20k dataset and the product categorical data. For each item node, we combined 5 one-hot encoded



vectors into a single feature vector, representing the classifications within the 5 commercial hierarchies discussed in Section 2.1.1. For each user node, we summed the feature vectors of its first nearest neighbours, i.e., accumulating the hierarchical classes of items that the user has purchased. Although this may seem unnecessary, it maintains the homogeneity of the graph such that user and item nodes share the same feature space.

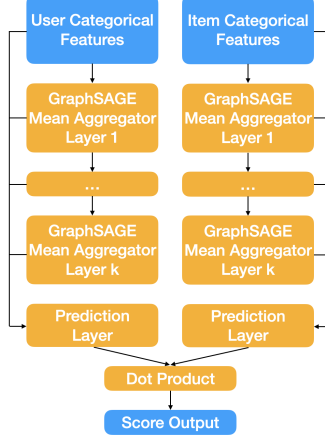


Figure 4.1: Graph Convolutional Network architecture with GraphSAGE mean aggregator for user and item categorical features input.

We trained this homogeneous graph using a 3-layer GraphSAGE model with the mean aggregator as illustrated in Figure 4.1. Unfortunately, the ranking performance underperformed when compared to NGCF and LightGCN. We suspect it is most likely due to the lack of meaningful features, including stronger features may boost the recommendation performance.

Still, this framework is intriguing that it can potentially solve the cold-start problem by obtaining explicit features from new users and new items beforehand. Since convolution on new unseen nodes can still be done as the learnable parameters are in the aggregators of message propagation layers.

### Product Description Features

In light of the previous attempt, we constructed a heterogeneous graph based on Tesco-20k dataset. Trainable embeddings are initialised for both user and item nodes, then we use the fastText [?] model to generate text representations from product description as item node features. We hope to maintain the original NGCF framework for learning embeddings while injecting additional item node information such as the semantic meaning of product description. As a result, we modified the NGCF model to include different weights in the user and item message propagation layers since the input dimensions of user and item will be different. We call this model Heterogeneous NGCF, as illustrated in Figure 4.2.

Upon evaluation, the ranking performance of 2-layer Heterogeneous NGCF was similar to ordinary 2-layer NGCF, suggesting the hybrid design of integrating features and embeddings did not make the model to extract extra signal from the graph.

This hybrid design was intended to separate the learning of node-specific signals and collabora-

tive signals within the embeddings. Since explicit features already cover node-specific information, we expected that the node embeddings would capture primarily collaborative signals. However, our evaluation indicates that either the explicit features did not contribute additional node-specific information, or the model architecture was not optimised to transform and aggregate representations from heterogeneous nodes.

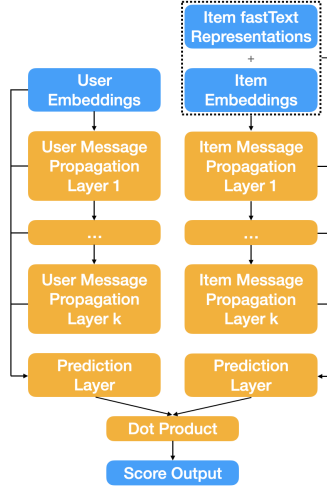


Figure 4.2: Heterogenous NGCF architecture for item fastText features input.

Based on the findings from these experiments, future work should focus on incorporating richer features for both user and item nodes to enhance model performance. The direction to separate and leverage node-specific and collaborative signals in heterogeneous graphs is also interesting, as it could lead to possible boost in ranking performance. Additionally, exploring advanced GNN frameworks that only allow feature inputs for user and item nodes could provide better solutions for addressing the cold-start problem. By refining feature integration and architecture design, there is significant potential for more robust and adaptive GNN-based recommendation systems.

## Chapter 5

# Conclusion

This study conducted a comprehensive experimental investigation into the application of Graph Neural Networks (GNNs) in recommendation systems, focusing on the Neural Graph Collaborative Filtering (NGCF) and Light Graph Convolutional Network (LightGCN) frameworks, along with their Graph SAmple and aggreGatE (GraphSAGE) variants. The primary objectives were to evaluate the performance of these models on real-world retail dataset, the quality of their graph embeddings, and their scalability with large-scale data.

Our experiments found that both optimally fine-tuned NGCF and LightGCN consistently outperformed the traditional Matrix Factorization (MF) model, with LightGCN demonstrating superior performance over NGCF in smaller scale graph. The performance of NGCF improved more significantly as the dataset scale increased, indicating the advanced modelling capabilities like NGCF is crucial when dealing with complex and large-scale data.

In terms of graph embeddings quality, we found that neither the item graph embeddings formed some kind of separation across food and non-food categories. The user graph embeddings formed a small cluster aside from the main blob, which cannot be differentiated based on user purchase behaviours by product categories. The sparsity analysis indicated that the graph embeddings were not well disentangled. While this does not necessarily imply that the GNN models were not learning, it suggests the interpretability of graphs embeddings is limited to some extent since the GNN-based models do not learn to clearly separate and cluster similar users and items by design, it is however possible to add constraints or fine-tune the loss objective for such behaviours.

We also demonstrated two effective strategies to improve the scalability of GNN-based models. The full inference method involves dividing the normalised Laplacian matrix into smaller sub-matrices, allowing for more manageable sub-products. This approach maintains model performance but incurs a slower computation time. On the other hand, the partial inference method involves GraphSAGE to sample and aggregate embeddings from a sub-graph, which preserves computational speed with even larger datasets but results in slight performance decline. Hypothetically, adapting partial inference methods for training and full inference methods for graph embeddings inference could be a solution to maintain both computational speed and model performance. We also experimented with two epoch definition strategies and suggested that better defined training loops and sampling techniques could potentially improve model convergence, especially in large-scale training.

Despite the promising results, our experiments also highlighted several areas for potential improvement. There is still room to optimise the model design to balance complexity, performance and computational efficiency. Future research should focus on refining GNN architectures and developing advanced sampling methods to further enhance scalability. Additionally, the cold-start problem remains a challenge, we propose a possible solution by incorporating richer and more diverse feature sets with heterogeneous GNNs to improve initial node representations.

In conclusion, our findings highlight the significant potential of GNN-based models in advancing recommendation systems and their large-scale applications. This study contributes to existing knowledge and provides practical insights for developing more efficient and accurate recommendation models, ultimately enhancing user experiences on digital platforms.

# Bibliography

- J. Bennett and S. Lanning. The netflix prize, 2007. URL <https://api.semanticscholar.org/CorpusID:9528522>.
- R. v. d. Berg, T. N. Kipf, and M. Welling. Graph convolutional matrix completion. *arXiv preprint arXiv:1706.02263*, 2017.
- P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov. Enriching word vectors with subword information. *arXiv preprint arXiv:1607.04606*, 2016.
- S. Brin and L. Page. The anatomy of a large-scale hypertextual web search engine. *Computer Networks*, 30:107–117, 1998. URL <http://www-db.stanford.edu/~backrub/google.html>.
- J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun. Spectral networks and locally connected networks on graphs. In *International Conference on Learning Representations (ICLR)*, 2014.
- J. Chen, H. Zhang, X. He, L. Nie, W. Liu, and T.-S. Chua. Attentive collaborative filtering: Multimedia recommendation with item- and component-level attention. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR ’17, page 335–344, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450350228. doi: 10.1145/3077136.3080797. URL <https://doi.org/10.1145/3077136.3080797>.
- A. Daigavane, B. Ravindran, and G. Aggarwal. Understanding convolutions on graphs. *Distill*, 2021. doi: 10.23915/distill.00032. <https://distill.pub/2021/understanding-gnns>.
- M. Defferrard, X. Bresson, and P. Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in Neural Information Processing Systems*, pages 3844–3852, 2016.
- T. Ebesu, B. Shen, and Y. Fang. Collaborative memory network for recommendation systems. In *The 41st International ACM SIGIR Conference on Research Development in Information Retrieval*, SIGIR ’18. ACM, jun 2018. doi: 10.1145/3209978.3209991. URL <http://dx.doi.org/10.1145/3209978.3209991>.
- H. Guo, J. Yu, Q. Liu, R. Tang, and Y. Zhang. Pal: a position-bias aware learning framework for ctr prediction in live recommender systems. In *Proceedings of the 13th ACM Conference on Recommender Systems*, RecSys ’19, page 452–456, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450362436. doi: 10.1145/3298689.3347033. URL <https://doi.org/10.1145/3298689.3347033>.

- W. Hamilton, Z. Ying, and J. Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, pages 1024–1034, 2017.
- W. L. Hamilton. Graph representation learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 14(3):1–159, 2020.
- X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua. Neural collaborative filtering, 2017. URL <https://arxiv.org/abs/1708.05031>.
- X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 639–648, 2020.
- N. P. Hurley and S. T. Rickard. Comparing measures of sparsity, 2009. URL <https://arxiv.org/abs/0811.4706>.
- Y. Ji, M. Yin, H. Yang, J. Zhou, V. W. Zheng, C. Shi, and Y. Fang. Accelerating large-scale heterogeneous interaction graph embedding learning via importance sampling. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 15:1 – 23, 2020. URL <https://api.semanticscholar.org/CorpusID:230525340>.
- T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representations (ICLR)*, 2017.
- Y. Koren. Factorization meets the neighborhood: a multifaceted collaborative filtering model. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '08, page 426–434, New York, NY, USA, 2008. Association for Computing Machinery. ISBN 9781605581934. doi: 10.1145/1401890.1401944. URL <https://doi.org/10.1145/1401890.1401944>.
- H. Lin, M. Yan, X. Ye, D. Fan, S. Pan, W. Chen, and Y. Xie. A comprehensive survey on distributed training of graph neural networks. *Proceedings of the IEEE*, 111:1572–1606, 2022. URL <https://api.semanticscholar.org/CorpusID:253447055>.
- Z. Liu, Y. Ma, Y. Ouyang, and Z. Xiong. Contrastive learning for recommender system, 2021. URL <https://arxiv.org/abs/2101.01317>.
- E. Mathieu, T. Rainforth, N. Siddharth, and Y. W. Teh. Disentangling disentanglement in variational autoencoders, 2019. URL <https://arxiv.org/abs/1812.02833>.
- L. McInnes, J. Healy, and J. Melville. Umap: Uniform manifold approximation and projection for dimension reduction, 2020. URL <https://arxiv.org/abs/1802.03426>.
- S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme. Bpr: Bayesian personalized ranking from implicit feedback, 2012. URL <https://arxiv.org/abs/1205.2618>.
- F. Ricci, L. Rokach, and B. Shapira, editors. *Recommender Systems Handbook*. Springer, 2nd edition, 2015. ISBN 978-1-4899-7636-9. doi: 10.1007/978-1-4899-7637-6. URL <https://link.springer.com/book/10.1007/978-1-4899-7637-6>.

- P. J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987. ISSN 0377-0427. doi: [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7). URL <https://www.sciencedirect.com/science/article/pii/0377042787901257>.
- X. Wang, X. He, M. Wang, F. Feng, and T.-S. Chua. Neural graph collaborative filtering. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '19. ACM, jul 2019. doi: 10.1145/3331184.3331267. URL <http://dx.doi.org/10.1145/3331184.3331267>.
- K. Xu, W. Hu, J. Leskovec, and S. Jegelka. How powerful are graph neural networks?, 2019. URL <https://arxiv.org/abs/1810.00826>.
- L. Yan, Z. Qin, H. Zhuang, X. Wang, M. Bendersky, and M. Najork. Revisiting two tower models for unbiased learning to rank. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, page 2410–2414, 2022.
- J.-H. Yang, C.-M. Chen, C.-J. Wang, and M.-F. Tsai. Hop-rec: High-order proximity for implicit recommendation. In *Proceedings of the 12th ACM Conference on Recommender Systems*, RecSys '18, pages 140–144, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450359016. doi: 10.1145/3240323.3240381. URL <https://doi.org/10.1145/3240323.3240381>.
- R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery Data Mining*, KDD '18. ACM, jul 2018. doi: 10.1145/3219819.3219890. URL <http://dx.doi.org/10.1145/3219819.3219890>.

# Appendix A

## Detailed Ranking Metrics

Table A.1: Full version of Tesco-20k test set ranking metrics of different models trained until convergence.

| Metric                     | MF (Baseline) | NGCF   | NGCF-SAGE | LightGCN | LightGCN-SAGE |
|----------------------------|---------------|--------|-----------|----------|---------------|
| <b>Recall@10</b>           | 0.0366        | 0.0374 | 0.0366    | 0.0384   | 0.0380        |
| <b>Recall@20</b>           | 0.0553        | 0.0580 | 0.0564    | 0.0604   | 0.0594        |
| <b>Recall@50</b>           | 0.0949        | 0.1018 | 0.0983    | 0.1073   | 0.1047        |
| <b>Recall@100</b>          | 0.1394        | 0.1523 | 0.1461    | 0.1606   | 0.1558        |
| <b>Recall@500</b>          | 0.3273        | 0.3492 | 0.3353    | 0.3691   | 0.3563        |
| <b>Rescaled Recall@10</b>  | 0.1526        | 0.1560 | 0.1525    | 0.1595   | 0.1583        |
| <b>Rescaled Recall@20</b>  | 0.1153        | 0.1211 | 0.1178    | 0.1260   | 0.1241        |
| <b>Rescaled Recall@50</b>  | 0.0962        | 0.1032 | 0.0996    | 0.1088   | 0.1061        |
| <b>Rescaled Recall@100</b> | 0.1394        | 0.1524 | 0.1461    | 0.1606   | 0.1558        |
| <b>Rescaled Recall@500</b> | 0.3273        | 0.3492 | 0.3353    | 0.3691   | 0.3563        |
| <b>Precision@10</b>        | 0.1526        | 0.1560 | 0.1525    | 0.1595   | 0.1583        |
| <b>Precision@20</b>        | 0.1153        | 0.1211 | 0.1178    | 0.1260   | 0.1241        |
| <b>Precision@50</b>        | 0.0792        | 0.0851 | 0.0821    | 0.0896   | 0.0874        |
| <b>Precision@100</b>       | 0.0582        | 0.0637 | 0.0611    | 0.0671   | 0.0651        |
| <b>Precision@500</b>       | 0.0274        | 0.0293 | 0.0281    | 0.0309   | 0.0299        |
| <b>AP@10</b>               | 0.0834        | 0.0845 | 0.0833    | 0.0867   | 0.0865        |
| <b>AP@20</b>               | 0.0496        | 0.0512 | 0.0501    | 0.0531   | 0.0527        |
| <b>AP@50</b>               | 0.0291        | 0.0308 | 0.0299    | 0.0324   | 0.0318        |
| <b>AP@100</b>              | 0.0324        | 0.0349 | 0.0336    | 0.0368   | 0.0360        |
| <b>AP@500</b>              | 0.0407        | 0.0444 | 0.0424    | 0.0475   | 0.0459        |
| <b>NDCG@10</b>             | 0.1832        | 0.1849 | 0.1826    | 0.1885   | 0.1881        |
| <b>NDCG@20</b>             | 0.1461        | 0.1500 | 0.1474    | 0.1545   | 0.1535        |
| <b>NDCG@50</b>             | 0.1196        | 0.1251 | 0.1221    | 0.1302   | 0.1282        |
| <b>NDCG@100</b>            | 0.1446        | 0.1536 | 0.1490    | 0.1603   | 0.1571        |
| <b>NDCG@500</b>            | 0.2315        | 0.2448 | 0.2367    | 0.2569   | 0.2500        |



Table A.2: Full version of Tesco-228k test set ranking metrics of different models trained for 10 epochs.

| Metric                     | MF (Baseline) | NGCF-5f | NGCF-SAGE | LightGCN-5f | LightGCN-SAGE |
|----------------------------|---------------|---------|-----------|-------------|---------------|
| <b>Recall@10</b>           | 0.0182        | 0.0203  | 0.0184    | 0.0193      | 0.0192        |
| <b>Recall@20</b>           | 0.0306        | 0.0343  | 0.0310    | 0.0328      | 0.0322        |
| <b>Recall@50</b>           | 0.0563        | 0.0646  | 0.0582    | 0.0620      | 0.0601        |
| <b>Recall@100</b>          | 0.0867        | 0.1001  | 0.0899    | 0.0963      | 0.0925        |
| <b>Recall@500</b>          | 0.2149        | 0.2522  | 0.2258    | 0.2425      | 0.2296        |
| <b>Rescaled Recall@10</b>  | 0.3930        | 0.4359  | 0.03961   | 0.4132      | 0.4147        |
| <b>Rescaled Recall@20</b>  | 0.3316        | 0.3693  | 0.3350    | 0.3530      | 0.3485        |
| <b>Rescaled Recall@50</b>  | 0.2457        | 0.2799  | 0.2531    | 0.2684      | 0.2617        |
| <b>Rescaled Recall@100</b> | 0.1898        | 0.2179  | 0.1963    | 0.2096      | 0.2023        |
| <b>Rescaled Recall@500</b> | 0.2153        | 0.2526  | 0.2262    | 0.2429      | 0.2300        |
| <b>Precision@10</b>        | 0.3930        | 0.4359  | 0.3961    | 0.4132      | 0.4147        |
| <b>Precision@20</b>        | 0.3316        | 0.3693  | 0.3350    | 0.3530      | 0.3485        |
| <b>Precision@50</b>        | 0.2457        | 0.2799  | 0.2531    | 0.2684      | 0.2617        |
| <b>Precision@100</b>       | 0.1898        | 0.2179  | 0.1963    | 0.2096      | 0.2023        |
| <b>Precision@500</b>       | 0.0955        | 0.1111  | 0.0998    | 0.1070      | 0.1018        |
| <b>MAP@10</b>              | 0.2631        | 0.3102  | 0.2673    | 0.2869      | 0.2884        |
| <b>MAP@20</b>              | 0.1921        | 0.2285  | 0.1952    | 0.2120      | 0.2104        |
| <b>MAP@50</b>              | 0.1159        | 0.1415  | 0.1198    | 0.1313      | 0.1283        |
| <b>MAP@100</b>             | 0.0751        | 0.0930  | 0.0782    | 0.0865      | 0.0835        |
| <b>MAP@500</b>             | 0.0504        | 0.0650  | 0.0537    | 0.0604      | 0.0566        |
| <b>NDCG@10</b>             | 0.4253        | 0.4730  | 0.4253    | 0.4475      | 0.4530        |
| <b>NDCG@20</b>             | 0.3710        | 0.4130  | 0.3721    | 0.3931      | 0.3929        |
| <b>NDCG@50</b>             | 0.2887        | 0.3262  | 0.2939    | 0.3116      | 0.3077        |
| <b>NDCG@100</b>            | 0.2295        | 0.2611  | 0.2348    | 0.2096      | 0.2447        |
| <b>NDCG@500</b>            | 0.2222        | 0.2578  | 0.2308    | 0.2501      | 0.2374        |

## Appendix B

# Detailed Embedding Analysis

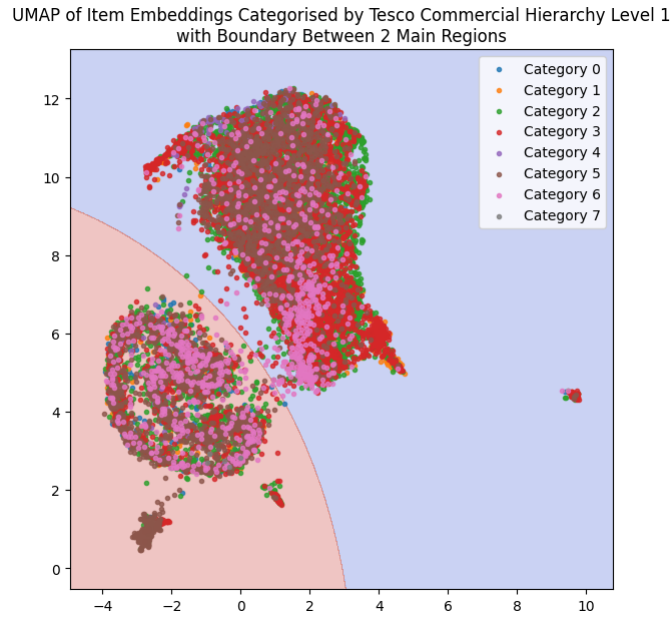


Figure B.1: UMAP of item embeddings of all categories from Tesco commercial hierarchy level 1, generated by LightGCN trained on Tesco-20k. The decision boundary of the two regions is learnt by a 2-component Gaussian Mixture Model.

Table B.1: Full version of ratio of UMAP item embeddings of all categories falling into red or blue region in Figure 3.6.

| Hierarchy Level 1<br>Category | Ratio      |             |
|-------------------------------|------------|-------------|
|                               | Red Region | Blue Region |
| 1                             | 0.2145     | 0.7855      |
| 2                             | 0.2667     | 0.7333      |
| 3                             | 0.2137     | 0.7863      |
| Food Categories               | 0.2413     | 0.7587      |
| 5                             | 0.4838     | 0.5162      |
| 6                             | 0.4374     | 0.5626      |
| Food Adjacent Categories      | 0.4727     | 0.5273      |
| 0                             | 0.7145     | 0.2855      |
| 4                             | 0.4587     | 0.5413      |
| 7                             | 1.0000     | 0.0000      |
| Non-food Categories           | 0.6830     | 0.3170      |

Table B.2: Mean sparsity values for user and item embeddings generated by LightGCN trained on Tesco-20k.

| Hierarchy Level 1<br>Category | Mean Sparsity |
|-------------------------------|---------------|
| 0                             | 0.1851        |
| 1                             | 0.1670        |
| 2                             | 0.1721        |
| 3                             | 0.1669        |
| 4                             | 0.1884        |
| 5                             | 0.1781        |
| 6                             | 0.1836        |
| 7                             | 0.2004        |

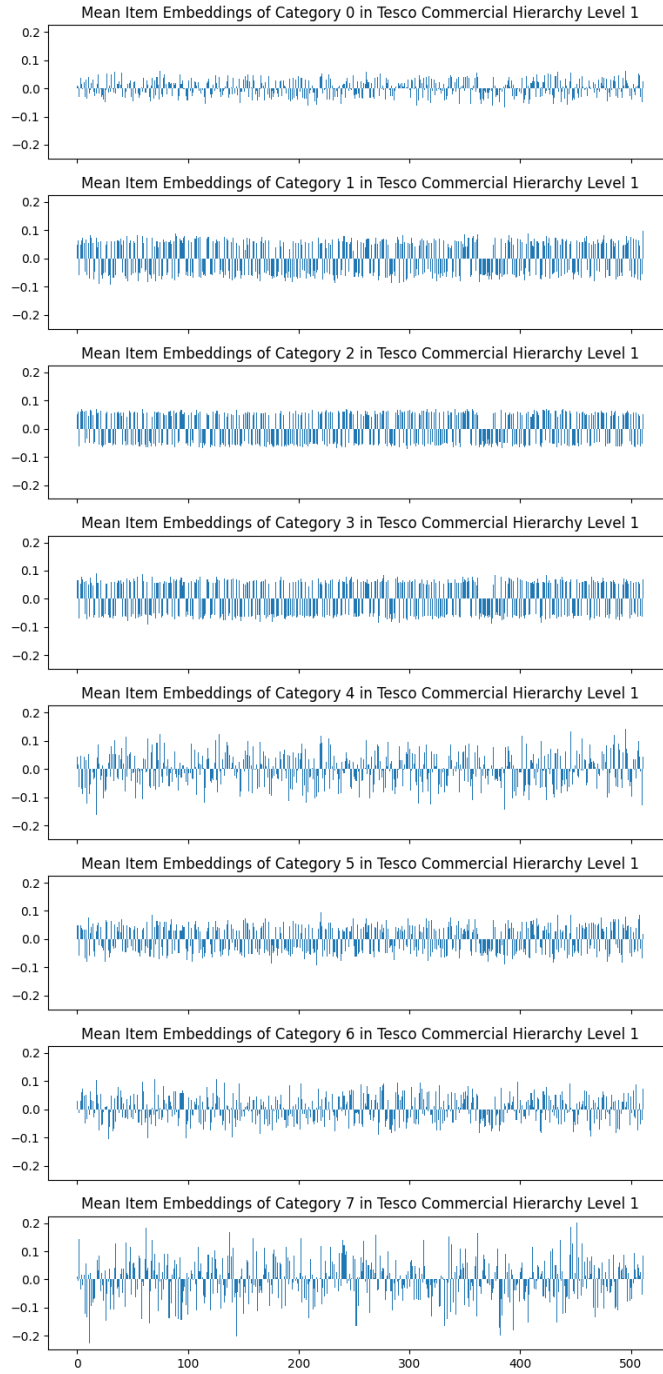


Figure B.2: Magnitude of each dimension in the mean item embeddings (512-d) of all categories from Tesco commercial hierarchy level 1, generated by LighGCN trained on Tesco-20k.